

Augusto Matheus Pinheiro Damasceno

**Proposta de implementação totalmente
paralela do algoritmo k-Nearest Neighbors
em FPGA**

Natal – RN

Dezembro de 2025

Augusto Matheus Pinheiro Damasceno

Proposta de implementação totalmente paralela do algoritmo k-Nearest Neighbors em FPGA

Trabalho de Conclusão de Curso de Engenharia de Computação da Universidade Federal do Rio Grande do Norte, apresentado como requisito parcial para a obtenção do grau de Bacharel em Engenharia de Computação

Orientador: Marcelo Augusto Costa
Fernandes

Universidade Federal do Rio Grande do Norte – UFRN
Departamento de Engenharia de Computação e Automação – DCA
Curso de Engenharia de Computação

Natal – RN
Dezembro de 2025

Universidade Federal do Rio Grande do Norte - UFRN
Sistema de Bibliotecas - SISBI
Catalogação de Publicação na Fonte. UFRN - Biblioteca Central Zila Mamede

Damasceno, Augusto Matheus Pinheiro.

Proposta de implementação totalmente paralela do algoritmo k-Nearest Neighbors em FPGA / Augusto Matheus Pinheiro Damasceno. - 2025.

69 f.: il.

Trabalho de Conclusão de Curso - TCC (graduação) - Universidade Federal do Rio Grande do Norte, Centro de Tecnologia, Curso de Engenharia de Computação, Natal, RN, 2025.

Orientação: Prof. Dr. Marcelo Augusto Costa Fernandes.

1. k-Nearest Neighbors - TCC. 2. FPGA - TCC. 3. Retrieval-Augmented Generation - TCC. 4. Busca Vetorial - TCC. 5. Aceleração em Hardware - TCC. 6. Large Language Models - TCC. I. Fernandes, Marcelo Augusto Costa. II. Título.

RN/UF/BCZM

CDU 004.383.8

Augusto Matheus Pinheiro Damasceno

Proposta de implementação totalmente paralela do algoritmo k-Nearest Neighbors em FPGA

Trabalho de Conclusão de Curso de Engenharia de Computação da Universidade Federal do Rio Grande do Norte, apresentado como requisito parcial para a obtenção do grau de Bacharel em Engenharia de Computação

Orientador: Marcelo Augusto Costa
Fernandes

Trabalho aprovado. Natal – RN, 11 de Dezembro de 2025:

Prof. Dr. Marcelo Augusto Costa Fernandes - Orientador
UFRN

Prof. Me. Mateus Arnaud Santos de Sousa Goldbarg
UFRN

Prof. Me. Pedro Victor Andrade Alves
UFRN

Natal – RN
Dezembro de 2025

*Dedico este trabalho à minha família, em especial aos meus ancestrais.
Eu sou porque vocês foram.*

AGRADECIMENTOS

Agradeço à minha família por ter proporcionado uma infância saudável e os fundamentos de formação, pois sempre farão parte da minha experiência.

Agradeço à Larissa pelo companheirismo e as várias manifestações do amor.

Agradeço ao meu orientador, Marcelo, que sempre acreditou no meu potencial e me apoiou.

*“Sem apego e sem aversão, o caminho é livre.”
(Ensino Budista)*

RESUMO

A crescente demanda por aplicações de Inteligência Artificial baseadas em Large Language Models (LLMs) tem impulsionado o desenvolvimento de técnicas como Retrieval-Augmented Generation (RAG), que combina recuperação de informações com geração de texto para reduzir alucinações e melhorar a precisão das respostas. No entanto, a etapa de busca vetorial em RAG apresenta um gargalo computacional significativo, especialmente em sistemas que requerem baixa latência e eficiência energética. Este trabalho propõe uma implementação totalmente paralela do algoritmo k-Nearest Neighbors (k-NN) em FPGA (Field-Programmable Gate Array) para aceleração de buscas em bancos de dados vetoriais. A arquitetura desenvolvida utiliza ponto fixo em vez de ponto flutuante para reduzir significativamente o consumo de recursos lógicos e a latência do sistema. A implementação foi realizada utilizando o Xilinx System Generator integrado ao MATLAB/Simulink, visando a FPGA Virtex-6 XC6VLX240T-1FFG1156 presente no kit ML605. O sistema é composto por módulos especializados para quantização, cálculo de score de similaridade por produto interno, e seleção dos k vizinhos mais próximos, explorando paralelismo massivo em hardware. Os resultados demonstram que a área ocupada pelo sistema foi baixa, possibilitando que o algoritmo seja escalado para processar um número maior de vetores. A latência total do sistema foi de 18,6 ns, tornando a solução viável para aplicações de inferência em tempo real de sistemas RAG.

Palavras-chave: k-Nearest Neighbors. FPGA. Retrieval-Augmented Generation. Busca Vetorial. Ponto Fixo. Aceleração em Hardware. Large Language Models. Embeddings. Baixa Latência. Eficiência Energética.

ABSTRACT

The growing demand for Artificial Intelligence applications based on Large Language Models (LLMs) has driven the development of techniques such as Retrieval-Augmented Generation (RAG), which combines information retrieval with text generation to reduce hallucinations and improve response accuracy. However, the vector search stage in RAG presents a significant computational bottleneck, especially in systems requiring low latency and energy efficiency. This work proposes a fully parallel implementation of the k-Nearest Neighbors (k-NN) algorithm on FPGA (Field-Programmable Gate Array) for accelerating searches in vector databases. The developed architecture uses fixed-point arithmetic instead of floating-point to significantly reduce logic resource consumption and system latency. The implementation was carried out using Xilinx System Generator integrated with MATLAB/Simulink, targeting the Virtex-6 XC6VLX240T-1FFG1156 FPGA present in the ML605 kit. The system comprises specialized modules for quantization, inner product similarity score calculation, and k-nearest neighbor selection, exploiting massive hardware parallelism. The results demonstrate that the system occupied low area, enabling the algorithm to be scaled to process a larger number of vectors. The total system latency was 18,6 ns, making the solution viable for real-time inference applications of RAG systems.

Keywords: k-Nearest Neighbors. FPGA. Retrieval-Augmented Generation. Vector Search. Fixed-Point. Hardware Acceleration. Large Language Models. Embeddings. Low Latency. Energy Efficiency.

LISTA DE ILUSTRAÇÕES

Figura 1 – RAG	21
Figura 2 – Kit de desenvolvimento Xilinx ML605 com FPGA Virtex-6	27
Figura 3 – Multiplicação no System Generator	29
Figura 4 – Bitonic Sort para 8 Elementos	31
Figura 5 – Arquitetura geral do sistema de busca k-NN em FPGA	34
Figura 6 – Módulo de Quantização	36
Figura 7 – Módulo de Quantização – Saturação	37
Figura 8 – Módulo Referência	39
Figura 9 – Módulo Score – Diagrama de blocos	40
Figura 10 – Diagrama de temporização – Acumulação e notificação de score	45
Figura 11 – Simulação do Módulo Score – Valores Teóricos vs Simulados	47
Figura 12 – Módulo de Controle da Ordenação	48
Figura 13 – Memória de Dados da Ordenação	49
Figura 14 – Unidade de Swap (Compare-and-Swap)	50

LISTA DE TABELAS

Tabela 1	– Métodos de busca em bancos de dados vetoriais	24
Tabela 2	– Formato IEEE 754 para float32 (32 bits)	28
Tabela 3	– Comparativo de desempenho: float32 vs ponto fixo UQ0.16	29
Tabela 4	– Conteúdo da ROM de Direção	51
Tabela 5	– Conteúdo da ROM de Seleção (Bitonic Index)	52
Tabela 6	– Utilização de recursos da FPGA Virtex-6 XC6VLX240T	54
Tabela 7	– Comparação de desempenhos: FPGA vs CPU	56
Tabela 8	– Parâmetros e avaliações de consumo de eletricidade para vários LLMs líderes	60

LISTA DE ABREVIATURAS E SIGLAS

ANN	<i>Approximate Nearest Neighbor</i>
BM25	<i>Best Matching 25</i>
DiskANN	<i>Disk-based Approximate Nearest Neighbor</i>
DSPy	<i>Declarative Self-Improving Python</i>
FAISS	<i>Facebook AI Similarity Search</i>
FPGA	<i>Field-Programmable Gate Array</i>
HNSW	<i>Hierarchical Navigable Small World</i>
IVF	<i>Inverted File Index</i>
IVFFlat	<i>Inverted File with Flat Compression</i>
KNN	<i>k-Nearest Neighbors</i>
LLM	<i>Large Language Model</i>
LSH	<i>Locality-Sensitive Hashing</i>
NGT	<i>Neighborhood Graph and Tree</i>
RAG	<i>Retrieval-Augmented Generation</i>
RRF	<i>Reciprocal Rank Fusion</i>
ScaNN	<i>Scalable Nearest Neighbors</i>
CO ₂	Dióxido de Carbono
IP Core	Intellectual Property core
AWS	Amazon Web Services
ASIC	Application-Specific Integrated Circuit
CLB	Configurable Logic Block
LC	Logic Cell
LE	Logic Element

ALM	Adaptive Logic Module
PIM	Processing-In-Memory
CPU	Central Processing Unit
DMA	Direct Memory Access
HDL	Hardware Description Languages
HLS	High-Level Synthesis
MSB	Most Significant Bit
LSB	Least Significant Bits
ROM	Read-Only Memory
LUT	Lookup Table
RAM	Random-access Memory

LISTA DE SÍMBOLOS

k	Número de vizinhos mais próximos no algoritmo k-NN
n	Dimensionalidade do vetor (número de elementos)
d	Dimensão do espaço vetorial
x	Vetor de entrada
r	Vetor de referência
x_i	i-ésimo elemento do vetor de entrada
r_i	i-ésimo elemento do vetor de referência
s	Score de similaridade (produto interno)
$\mathbf{x}$	Notação vetorial para vetor de entrada
$\mathbf{r}$	Notação vetorial para vetor de referência
z^{-1}	Operador de delay (atraso de um ciclo de clock)
f_{clk}	Frequência de clock
T_{clk}	Período de clock
$T_{latency}$	Latência total do sistema
N_{cycles}	Número de ciclos de clock

SUMÁRIO

1	INTRODUÇÃO	17
1.1	Trabalhos Relacionados	18
1.2	Objetivos	19
1.2.1	Objetivos Específicos	19
1.3	Estrutura do Trabalho	19
2	FUNDAMENTAÇÃO TEÓRICA	21
2.1	Arquitetura RAG	21
2.2	Busca Vetorial	22
2.2.1	Algoritmos Utilizados em Sistemas RAG	23
2.2.2	Predominância de HNSW em Software	25
2.2.3	Considerações para Implementação em Hardware	25
2.3	FPGA	26
2.3.1	Plataforma de Desenvolvimento	26
2.4	O Padrão IEEE 754	28
2.5	Ponto Fixo vs Ponto Flutuante	29
2.5.1	Exemplo de Quantização	29
2.6	Cálculo de Produto Interno	30
2.6.1	Decomposição do Cálculo	30
2.7	Algoritmo de Ordenação Bitônica	31
3	PROPOSTA DE IMPLEMENTAÇÃO	32
3.1	Visão Geral da Arquitetura	32
3.1.1	Escopo de Processamento	34
3.2	Dados de Entrada	34
3.3	Tipo Numérico e Dimensionalidade	34
3.3.1	Análise Estatística dos Dados	35
3.4	Módulo de Quantização	35
3.4.1	Design da Quantização	36
3.4.2	Vantagens da Abordagem	38
3.5	Módulo Referência	38
3.6	Módulo Score	40
3.6.0.1	Processamento por Ciclo	40
3.6.0.2	Modo de Operação em Streaming	41
3.6.1	Sincronização com o Módulo Referência	41

3.6.2	Pipeline e Latência do Módulo	42
3.6.3	Formato de Dados e Evolução da Precisão	42
3.6.4	Acumulação de Scores Parciais e Sincronização com o Módulo Ordenação .	43
3.6.5	Desempenho e Escalabilidade	45
3.6.6	Teste em Ambiente de Simulação	46
3.7	Módulo de Ordenação	47
3.7.1	Arquitetura do Módulo	48
3.7.2	Funcionamento Ciclo a Ciclo	50
3.7.3	Operação de Swap	51
3.7.4	Conteúdo das ROMs	51
3.7.4.1	ROM de Direção	51
3.7.4.2	ROM de Seleção (Bitonic Index)	51
3.7.5	Desempenho e Características	52
3.7.6	Vantagens da Implementação em Hardware	52
3.8	Saída	52
4	RESULTADOS E ANÁLISE	53
4.1	Análise de Temporização	53
4.1.1	Resultados de Temporização	53
4.1.2	Caminho Crítico	53
4.2	Utilização de Recursos	54
4.2.1	Análise dos Recursos Críticos	54
4.2.1.1	DSP48E1 (16,9% - 130 de 768 blocos)	54
4.2.1.2	RAMB18E1 (3,1% - 26 de 832 blocos)	55
4.2.1.3	Lógica Reconfigurável (3,7% LUTs - 5.580 de 150.720)	55
4.2.1.4	Registradores (0,4% - 1.201 de 301.440)	55
4.2.1.5	IOBs (87,2% - 523 de 600 pinos)	55
4.2.1.6	Recursos de Clock (3,1% - 1 BUFG de 32)	55
4.3	Desempenho Computacional	56
4.3.1	Latência e Throughput	56
4.3.2	Comparação Hardware vs Software	56
4.4	Sumário dos Resultados	56
4.5	Contribuições	57
5	CONCLUSÃO	58
5.1	Limitação Identificada e Trabalhos Futuros	58
5.2	Considerações Finais	58

APÊNDICES	59
APÊNDICE A – CONSUMO ENERGÉTICO DE LARGE LANGUAGE MODELS	60
APÊNDICE B – ANÁLISE ESTATÍSTICA DOS DATASETS DA COHERE	62
REFERÊNCIAS	68

1 INTRODUÇÃO

O interesse público sobre Inteligência Artificial e *chats* baseados em Large Language Models (LLM) cresceu exponencialmente desde o final de 2022 com o lançamento do ChatGPT. Isso refletiu no mercado financeiro, onde houve grande valorização de empresas da área de Inteligência Artificial (IA) e de tecnologias como design de hardware de chips para aceleração de algoritmos da Nvidia e litografia da ASML.

Segundo (JI; JIANG, 2026), a grandiosidade dos LLMs se manifesta em três dimensões: a escala imensa dos parâmetros do modelo, a abundância de dados e as formidáveis demandas computacionais. A enorme demanda computacional tem como consequências o alto consumo energético e a emissão de CO_2 nas fontes geradoras de energia. O treinamento do GPT-3 consumiu aproximadamente 1287 MWh, acompanhado por mais de 552 toneladas de emissões de carbono, enquanto o GPT-4 requer mais de 40 vezes essa quantidade de eletricidade. Além do consumo energético, a projeção de consumo anual de água apenas para a inferência do modelo GPT-4o em 2025 pode chegar a 1.579.680 quilolitros no cenário de maior utilização, um volume de água doce evaporada equivalente à necessidade anual de bebida de aproximadamente 1,2 milhão de pessoas (JEGHAM et al., 2025). Dados detalhados sobre o consumo energético e emissões de CO_2 de diversos LLMs podem ser consultados no Apêndice A.

Para otimizar algoritmos são estudadas estruturas de dados adequadas, aperfeiçoamento de localidade de referência (uso eficiente de cache), técnicas avançadas de compilação, fluxo e dependência de dados para viabilizar paralelismo ou computação distribuída. Contudo, existem cenários em que, mesmo aplicadas estas técnicas, o processamento permanece excessivamente oneroso em termos de tempo e energia. Uma solução para este problema é a aceleração em hardware, comumente realizada em Field-Programmable Gate Array (FPGA). A partir do design validado em FPGA, podem ser criados núcleos de propriedade intelectual (*Intellectual Property cores* – IP Core) para serem processados em FPGA, por exemplo: as instâncias EC2 F1 da Amazon Web Services (AWS) (Amazon Web Services, 2025), ou serem criados chips dedicados, chamados *Application-Specific Integrated Circuit* (ASIC).

As FPGAs são dispositivos programáveis contendo campos repetidos de pequenos blocos lógicos e elementos (MEYER-BAESE, 2014). Estes blocos são denominados *slice* ou *Configurable Logic Block* (CLB) pela Xilinx, *Logic Cell* (LC), *Logic Element* (LE) ou *Adaptive Logic Module* (ALM) pela Altera.

Definido o problema do custo de recursos de tempo e energia, e uma solução eficaz para redução de consumo destes recursos, foi escolhida uma técnica para ser acelerada em hardware: o Retrieval-Augmented Generation (RAG), publicado em (LEWIS et al., 2020). RAG é uma das técnicas mais importantes na área de LLMs e objetiva melhorar o funcionamento de LLMs em relação à:

- **Alucinações¹**;
- Limitações de Acesso ao Conhecimento;
- Revisão e expansão de memória.

RAG consiste em utilizar uma base de dados externa aos dados em que a LLM foi treinada, como a base de conhecimento de uma organização, para enriquecer um *prompt*, otimizando assim a resposta da LLM. São utilizados diversos tipos de bancos de dados, sendo o vetorial o mais comum, onde a busca é realizada através de comparação entre vetores numéricos de alta dimensão chamados *embeddings* (detalhes no Capítulo 2). Este processo de busca em banco de dados vetoriais acelerado em FPGA é o objetivo deste trabalho.

1.1 Trabalhos Relacionados

Em (JUNG et al., 2025) é implementada aceleração do cálculo de similaridade vetorial usando *Processing-In-Memory* (PIM), porém em FPGA é processado apenas o valor atribuído à similaridade entre vetores (distância euclidiana e produto escalar), o restante do algoritmo é processado em *Central Processing Unit* (CPU). O presente trabalho propõe uma implementação com todo o processamento do algoritmo em hardware.

Em (ABDELRASOUL; SHABAN; ABDEL-KADER, 2021) foi realizado estudo comparativo de algoritmos de ordenação implementados em FPGA, avaliando arquiteturas síncronas e pipeline. Para arquiteturas pipeline, concluiu que o Bitonic Merge Sort apresenta excelente desempenho em tempo de execução e, embora seja ligeiramente maior em área que o Odd-Even Merge Sort, é preferido por projetistas devido à sua estrutura altamente regular, facilitando síntese e roteamento em hardware. Esta conclusão fundamenta a escolha do Bitonic Sort para o módulo de ordenação do presente trabalho.

¹A alucinação da IA é um fenômeno em que um grande modelo de linguagem (LLM) percebe padrões inexistentes, criando saídas imprecisas ou sem sentido (IBM, 2024).

Em (DIAS et al., 2021) a capacidade da FPGA Virtex-6 xc6vlx240t em suportar arquiteturas de alto desempenho com paralelismo massivo é evidenciada.

1.2 Objetivos

Projetar, implementar e validar uma arquitetura totalmente paralela do algoritmo k-Nearest Neighbors (k-NN) em FPGA utilizando aritmética de ponto fixo, visando baixa latência e eficiência de recursos para aceleração de buscas vetoriais em sistemas de Retrieval-Augmented Generation (RAG).

1.2.1 Objetivos Específicos

- Projetar a arquitetura digital do módulo utilizando o software *System Generator* (Xilinx Inc., 2012c) com a ferramenta Simulink (The MathWorks Inc., 2024b) do MATLAB versão 2012b (The MathWorks Inc., 2024a), com foco em paralelismo e pipelining;
- Implementar o módulo em FPGA utilizando o *System Generator* e realizar a integração com um sistema de teste que contém vetores de entrada e resultados esperados;
- Validar a funcionalidade e a precisão do módulo, garantindo que os vizinhos mais próximos sejam corretamente identificados em diferentes cenários;
- Avaliar o desempenho em termos de latência, uso de recursos lógicos e eficiência energética e comparar com abordagens sequenciais ou em CPU utilizando o software ISE 14.3 (Xilinx Inc., 2012a).

1.3 Estrutura do Trabalho

Este trabalho está organizado da seguinte forma:

O Capítulo 1 (Introdução) apresenta o contexto da crescente demanda computacional de Large Language Models (LLMs), introduz a técnica RAG e justifica a necessidade de aceleração em hardware para buscas vetoriais.

O Capítulo 2 (Fundamentação Teórica) apresenta a fundamentação teórica sobre arquiteturas RAG e algoritmos de busca vetorial, discutindo a dicotomia entre a eficiência do *Hierarchical Navigable Small World* (HNSW) em software e a adequação do k-NN para aceleração em hardware, além de detalhar as especificações da plataforma FPGA (Xilinx ML605) utilizada no projeto.

O Capítulo 3 (Proposta de Implementação) apresenta a arquitetura completa do acelerador hardware k-NN, detalhando seus quatro módulos principais: Quantização, Referência, Score e Ordenação.

O Capítulo 4 (Resultados e Análise) apresenta os resultados da síntese e implementação na FPGA Xilinx Virtex-6, incluindo análise de temporização, utilização de recursos, desempenho computacional e potencial de escalabilidade.

O Capítulo 5 (Conclusão) apresenta as principais contribuições do trabalho, confirma a adequação do k-NN para aceleração hardware, discute as limitações identificadas e apresenta perspectivas para trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Arquitetura RAG

A arquitetura de um sistema utilizando RAG pode ser vista na Figura 1.

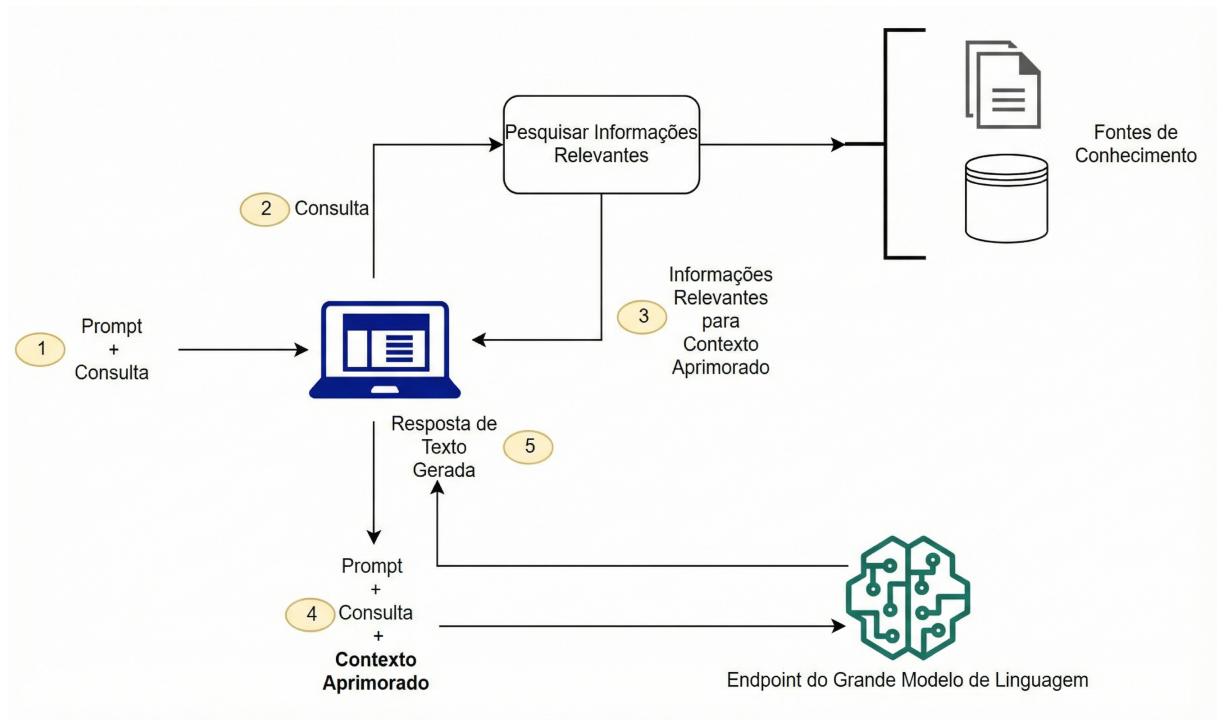


Figura 1 – Fluxo conceitual de uso do RAG com LLMs

Fonte: (Amazon Web Services, 2024)

No processo de inferência de um LLM, os dados de entrada são tokenizados (reduzidos a partes menores, como subpalavras ou caracteres) e em seguida cada token é transformado em um vetor numérico com alta dimensão chamado *embedding*. Os embeddings inicialmente representam relações semânticas e sintáticas aprendidas durante a fase de treinamento do LLM. Em seguida, os *embeddings* são modificados em um processo chamado de autoatenção para representar sua relação com os outros *embeddings* dos demais tokens, dentro do contexto dos dados de entrada. O RAG enriquece o *prompt* gerando os embeddings a partir dos dados de entrada e buscando em banco de dados os documentos mais relevantes, ou seja, os mais próximos dos embeddings da entrada. São utilizados diversos tipos de bancos de dados, sendo o vetorial o mais comum; um exemplo de busca pode ser visto no Código 1.

```
1 query = 'Who founded Youtube'
2 response = co.embed(texts=[query], model='multilingual-22-12')
3 query_embedding = response.embeddings
4 query_embedding = torch.tensor(query_embedding)
5 dot_scores = torch.mm(query_embedding, doc_embeddings.transpose
    (0, 1))
6 top_k = torch.topk(dot_scores, k=3)
```

Fonte: (Cohere AI, 2023b)

Código 1 – Exemplo de Busca com Embeddings da Cohere e Dataset wikipedia-22-12-en-embeddings

2.2 Busca Vetorial

A busca em bancos de dados vetoriais pode ser realizada através de diferentes abordagens, cada uma com trade-offs específicos entre acurácia, velocidade e escalabilidade:

- **Busca Exata (k-NN - k-Nearest Neighbors):** Algoritmo que identifica os k vetores mais próximos a um vetor de consulta através do cálculo exaustivo de similaridade (como distância euclidiana ou produto escalar) entre a consulta e todos os vetores do banco de dados. O algoritmo executa os seguintes passos:
 1. Calcula a similaridade entre o vetor de consulta e cada vetor do banco de dados;
 2. Ordena os vetores com base na similaridade calculada;
 3. Retorna os k vetores mais próximos com base na ordem de similaridade.

Este método garante acurácia de 100% (recall perfeito) ao custo de complexidade temporal $\mathcal{O}(n \cdot d)$, onde n é o número de vetores e d a dimensionalidade, e complexidade espacial $\mathcal{O}(k)$ para armazenar os resultados intermediários. Suas principais vantagens incluem:

- **Determinismo absoluto:** Sempre retorna exatamente os k vizinhos mais próximos;
- **Simplicidade de implementação:** Não requer estruturas de dados complexas ou fase de treinamento;
- **Paralelização trivial:** Cálculos de similaridade são completamente independentes;
- **Acesso sequencial à memória:** Permite streaming eficiente e otimização de cache;
- **Latência previsível:** Tempo de execução constante e determinístico.

- **Busca Aproximada (ANN - Approximate Nearest Neighbors):** Utiliza estruturas de indexação como *Inverted File Index* (IVF) e HNSW para reduzir a complexidade para $\mathcal{O}(\log n)$, sacrificando acurácia (tipicamente 90-98% de recall) em troca de velocidade. Embora seja mais eficiente em software para bases massivas, apresenta desvantagens para implementação em hardware:
 - **Acesso aleatório à memória:** Navegação por grafos ou árvores gera latências imprevisíveis;
 - **Complexidade estrutural:** Requer gerenciamento dinâmico de ponteiros e índices;
 - **Dependências de dados:** Limita paralelização por introduzir dependências entre operações;
 - **Variabilidade de latência:** Tempo de execução varia conforme estrutura do índice.

2.2.1 Algoritmos Utilizados em Sistemas RAG

A Tabela 1 apresenta os principais sistemas de RAG e os algoritmos de busca vetorial que eles empregam.

Sistema	Algoritmo de Busca Vetorial
Amazon Kendra ¹	Inverted File with Flat Compression (IVFFlat)
Azure Cognitive Search ²	Híbrido com KNN e HNSW
ChromaDB ³	HNSW
DSPy ⁴	Qualquer LLM e Busca
Elasticsearch ⁵	Best Matching 25 (BM25), RRF, HNSW, KNN
FAISS ⁶	IVF, HNSW
Google Vertex AI ⁷	ScaNN (Scalable Nearest Neighbors), BM25, Reciprocal Rank Fusion (RRF)
Haystack ⁸	Pinecone, Weaviate, Milvus, etc.
LangChain ⁹	Pinecone, Weaviate, Qdrant, Milvus, FAISS, etc.
LlamaIndex ¹⁰	Pinecone, Weaviate, Qdrant, Milvus, etc.
Milvus ¹¹	HNSW, IVF
MongoDB Atlas ¹²	HNSW
OpenAI GPT-4 ¹³	Não Especifica
Pinecone ¹⁴	KNN + ANN
Qdrant ¹⁵	HNSW, DiskANN
Redis Stack ¹⁶	HNSW
SurrealDB ¹⁷	HNSW
Vald ¹⁸	Neighborhood Graph and Tree (NGT), HNSW
Vespa ¹⁹	HNSW, Locality-Sensitive Hashing (LSH)
Weaviate ²⁰	Híbrido com HNSW, BM25 e RRF

Tabela 1 – Métodos de busca em bancos de dados vetoriais

¹(Amazon Web Services, 2023)

²(Microsoft, 2023)

³(Chroma, 2023)

⁴(Stanford University, 2023)

⁵(Elastic, 2023)

⁶(Facebook AI, 2023)

⁷(Google Cloud, 2023)

⁸(Deepset, 2023)

⁹(LangChain, 2023)

¹⁰(LlamaIndex (formerly GPT Index), 2023)

¹¹(Zilliz, 2023)

¹²(MongoDB Inc., 2023)

¹³(OpenAI, 2023)

¹⁴(Pinecone Systems Inc., 2023)

¹⁵(Qdrant, 2023)

¹⁶(Redis Inc., 2023)

2.2.2 Predominância de HNSW em Software

A análise da Tabela 1 revela a predominância do algoritmo HNSW (presente em 15 dos 20 sistemas, implicitamente em Haystack, LangChain e LlamaIndex), justificada por:

1. **Complexidade logarítmica:** $\mathcal{O}(\log n)$ vs. $\mathcal{O}(n)$ do k-NN, essencial para bases com milhões/bilhões de vetores;
2. **Escalabilidade:** Capacidade de lidar com crescimento massivo de dados sem degradação proporcional de desempenho;
3. **Acurácia aceitável:** Embora não garanta 100% de acurácia, algoritmos ANN como HNSW oferecem recall@10 superior a 95% em benchmarks padrão (Facebook AI, 2023; Pinecone Systems Inc., 2023), considerado suficiente para recuperação de contexto em aplicações RAG;
4. **Trade-off favorável:** Redução de 10-100× no tempo de busca com perda mínima de qualidade.

Essa preferência por algoritmos ANN em software se justifica pelas restrições computacionais e de tempo de resposta exigidas por aplicações RAG em larga escala, onde a busca exaustiva (k-NN) se torna proibitiva para bases com centenas de milhões de vetores.

2.2.3 Considerações para Implementação em Hardware

No contexto de aceleração em FPGA, entretanto, o cenário muda significativamente. Aspectos críticos para a escolha do algoritmo incluem:

- **Padrão de acesso à memória:** k-NN permite acesso sequencial e streaming, ideal para Direct Memory Access (DMA); HNSW requer acesso aleatório ao grafo, gerando latências imprevisíveis;
- **Paralelismo:** k-NN possui independência total entre cálculos de similaridade, permitindo paralelização massiva; HNSW tem dependências de dados que limitam exploração de paralelismo;
- **Previsibilidade:** k-NN oferece latência determinística; HNSW varia conforme a estrutura do grafo e ponto de entrada;

¹⁷(SurrealDB, 2023)

¹⁸(Vald Community, 2023)

¹⁹(Yahoo Inc., 2023)

²⁰(SeMI Technologies, 2023)

- **Complexidade de implementação:** k-NN requer apenas aritmética de ponto fixo e ordenação; HNSW exige gerenciamento complexo de ponteiros e estruturas de dados dinâmicas.

Essas características tornam o k-NN mais adequado para aceleração em hardware, conforme analisado na seção seguinte e corroborado por (JUNG et al., 2025), que demonstrou que transferências de memória contínuas são fundamentais para eficiência em FPGA.

2.3 FPGA

Field-Programmable Gate Array (FPGA) é um circuito integrado que pode ser configurado pelo usuário após a fabricação, ao contrário de ASICs (*Application-Specific Integrated Circuits*) que têm sua funcionalidade definida durante a manufatura. Esta característica permite que FPGAs sejam reprogramadas para implementar diferentes arquiteturas de hardware, tornando-as ideais para prototipagem rápida, aceleração de algoritmos e aplicações que requerem paralelismo massivo.

2.3.1 Plataforma de Desenvolvimento

A implementação e validação deste trabalho foram realizadas utilizando o **Xilinx ML605 Evaluation Kit**, uma plataforma de desenvolvimento profissional baseada na FPGA Virtex-6 XC6VLX240T-1FFG1156, mostrada na Figura 2.

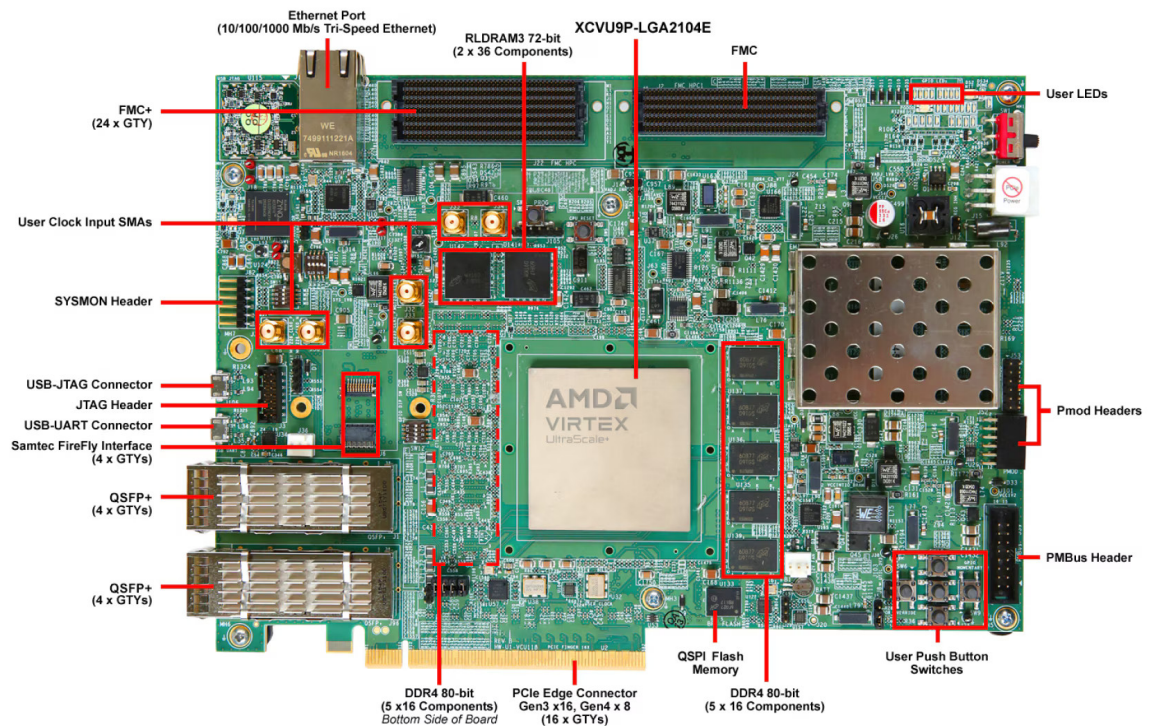


Figura 2 – Kit de desenvolvimento Xilinx ML605 com FPGA Virtex-6

Fonte: Xilinx Inc. (Xilinx Inc., 2012b)

O kit ML605 oferece recursos essenciais para desenvolvimento de sistemas de alto desempenho:

- **FPGA Virtex-6 XC6VLX240T:**
 - 150.720 LUTs (Look-Up Tables)
 - 301.440 Flip-Flops
 - 768 blocos DSP48E1
 - 832 blocos BRAM de 36 Kbits (14,976 Mbits totais)
 - 600 pinos de I/O de usuário
- **Memória externa:**
 - 512 MB DDR3 SODIMM (expansível até 1 GB)
 - 128 MB Flash Linear para configuração
- **Interfaces de comunicação:**
 - PCIe x8 Gen1 (2,5 GT/s por lane)
 - Gigabit Ethernet 10/100/1000

- USB 2.0 para JTAG e UART
- HDMI, DisplayPort
- **Clock e sincronização:**
 - Oscilador de 200 MHz (pode ser configurado para 50/100/200 MHz)
 - Suporte a múltiplos domínios de clock

Esta plataforma foi escolhida por oferecer recursos suficientes para implementação e validação do algoritmo k-NN proposto, além de ferramentas de desenvolvimento maduras (ISE Design Suite) e ampla documentação técnica disponível.

2.4 O Padrão IEEE 754

Os vetores de embeddings utilizados em sistemas RAG são tipicamente armazenados e transmitidos no formato de ponto flutuante de precisão simples (float32), conforme definido pelo padrão IEEE 754 (IEEE, 2019). Este formato representa números reais usando 32 bits, divididos em três campos: sinal (1 bit), expoente (8 bits) e mantissa (23 bits), permitindo expressar uma ampla faixa de valores com precisão adequada para aplicações de aprendizado de máquina.

No contexto deste trabalho, os embeddings de entrada do sistema chegam neste formato float32, sendo posteriormente convertidos para representação em ponto fixo para processamento eficiente na FPGA. A compreensão desta representação é essencial para implementar corretamente a interface de entrada do acelerador e garantir que os dados sejam interpretados adequadamente durante a conversão.

A disposição dos bits do padrão IEEE 754 pode ser vista na Tabela 2 e a equação para se chegar ao valor real na Equação 2.1, conforme definido na seção 3.4 (*Binary interchange format encodings*) de IEEE (2019).

Campo	Sinal (S)	Expoente (E)	Fração/Mantissa (M)
Bits	31	30–23	22–0
Quantidade	1 bit	8 bits	23 bits

Tabela 2 – Formato IEEE 754 para float32 (32 bits)

Fonte: Adaptado de (IEEE, 2019), Seção 3.4

$$\text{Valor_Real} = (-1)^S \times \left(1 + \sum_{i=1}^{23} b_{23-i} \times 2^{-i} \right) \times 2^{(E-127)} \quad (2.1)$$

2.5 Ponto Fixo vs Ponto Flutuante

O processamento aritmético de ponto flutuante tem custo de tempo e área extremamente maior do que ponto fixo à medida que o sistema é escalado. Na Tabela 3 está demonstrado o comparativo de desempenho de dois sistemas que realizam uma simples multiplicação de dois valores (Figura 3), sendo um em float32 e outro em ponto fixo UQ0.16, para a FPGA Virtex-6 XC6VLX240T-1FFG1156:

Tipo de Dado	Slice Registers	Slice LUTs	DSP48E1	Latência
Float32	55	95	3	4.263 ns
Ponto Fixo UQ0.16	0	0	1	2.116 ns

Tabela 3 – Comparativo de desempenho: float32 vs ponto fixo UQ0.16

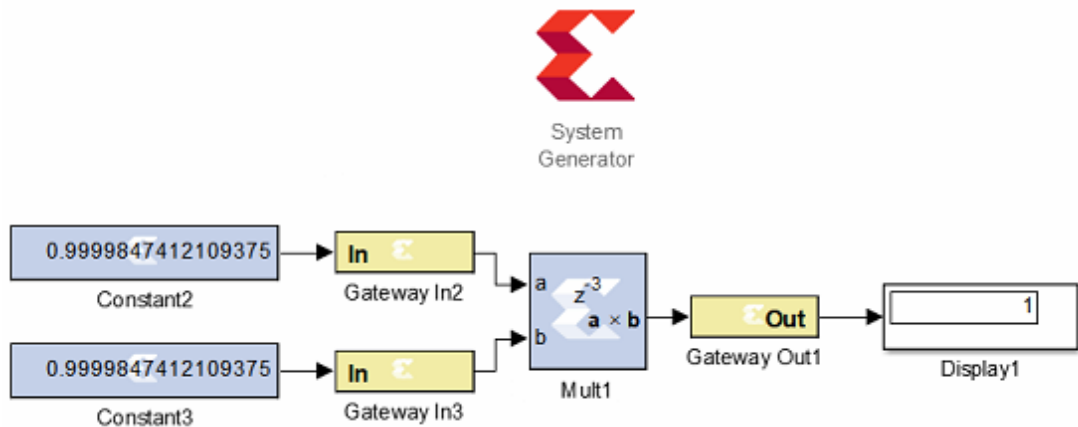


Figura 3 – Multiplicação no System Generator

Os resultados demonstram uma redução de 100% em registradores e LUTs, 66,7% em blocos DSP e 50,4% em latência ao utilizar ponto fixo em vez de ponto flutuante. Esta diferença se torna ainda mais significativa em sistemas com múltiplas operações aritméticas paralelas.

2.5.1 Exemplo de Quantização

Para ilustrar o processo de conversão, considere o valor $-0,5$ presente em um embedding de entrada. A quantização para o formato UQ0.16 utilizado neste trabalho ocorre em duas etapas:

Etapas 1 - Normalização: Aplicando a transformação descrita anteriormente (offset de 2 e multiplicação por 0,25) para mapear o intervalo $[-2, 2]$ em $[0, 1]$:

$$V_{\text{norm}} = (-0,5 + 2) \times 0,25 = 1,5 \times 0,25 = 0,375 \quad (2.2)$$

Etapa 2 - Conversão para Ponto Fixo UQ0.16: O valor normalizado é convertido para inteiro multiplicando-o por 2^{16} e truncando:

$$V_{\text{UQ0.16}} = \lfloor 0,375 \times 2^{16} \rfloor = \lfloor 0,375 \times 65536 \rfloor = 24576 \quad (2.3)$$

Em representação hexadecimal: 0x6000

Verificação: Para recuperar o valor aproximado, divide-se por 2^{16} :

$$V_{\text{recuperado}} = \frac{24576}{65536} = 0,375 \quad (2.4)$$

E revertendo a normalização:

$$V_{\text{original}} = \frac{0,375}{0,25} - 2 = 1,5 - 2 = -0,5 \quad (2.5)$$

Este exemplo demonstra que a quantização preserva perfeitamente valores que podem ser representados exatamente em 16 bits fracionários, com erro de quantização desprezível para a maioria dos valores práticos em embeddings.

2.6 Cálculo de Produto Interno

O produto interno entre vetores de 1024 dimensões é computado através de uma estratégia de paralelização temporal que equilibra throughput e utilização de recursos. O cálculo completo é dividido em 64 iterações, processando 16 elementos por ciclo de clock, conforme ilustrado na Figura 9.

2.6.1 Decomposição do Cálculo

Sejam $\mathbf{x} = (x_0, x_1, \dots, x_{1023})$ o vetor de entrada e $\mathbf{r} = (r_0, r_1, \dots, r_{1023})$ o vetor de referência. O produto interno total é definido como:

$$\langle \mathbf{x}, \mathbf{r} \rangle = \sum_{i=0}^{1023} x_i \times r_i \quad (2.6)$$

Este somatório é decomposto em 64 blocos de 16 elementos cada, permitindo o cálculo em streaming:

$$\langle \mathbf{x}, \mathbf{r} \rangle = \sum_{k=0}^{63} \underbrace{\sum_{j=0}^{15} x_{16k+j} \times r_{16k+j}}_{\text{Score}_{\text{parcial}}[k]} \quad (2.7)$$

onde $\text{Score}_{\text{parcial}}[k]$ representa a contribuição do bloco k para o produto interno total.

2.7 Algoritmo de Ordenação Bitônica

A rede de ordenação bitônica é um algoritmo paralelo ideal para implementação em hardware devido às seguintes características:

- **Número fixo de comparações:** Para N elementos, requer $\frac{N \log^2(N)}{2}$ comparações organizadas em $\log^2(N)$ estágios;
- **Padrão determinístico:** A sequência de comparações é conhecida antecipadamente e pode ser armazenada em ROMs;
- **Paralelismo intrínseco:** Múltiplas comparações podem ocorrer simultaneamente em cada estágio;
- **Regularidade estrutural:** Facilita a implementação e verificação em HDL.

Para o caso específico de 8 vetores ($N = 8$), a rede de ordenação requer:

$$\text{Estágios} = \frac{\log_2(N) \times (\log_2(N) + 1)}{2} = \frac{3 \times 4}{2} = 6 \quad (2.8)$$

Na Figura 4 é ilustrada a estrutura da rede de ordenação bitônica para 8 elementos, demonstrando os pares de comparações em cada estágio.

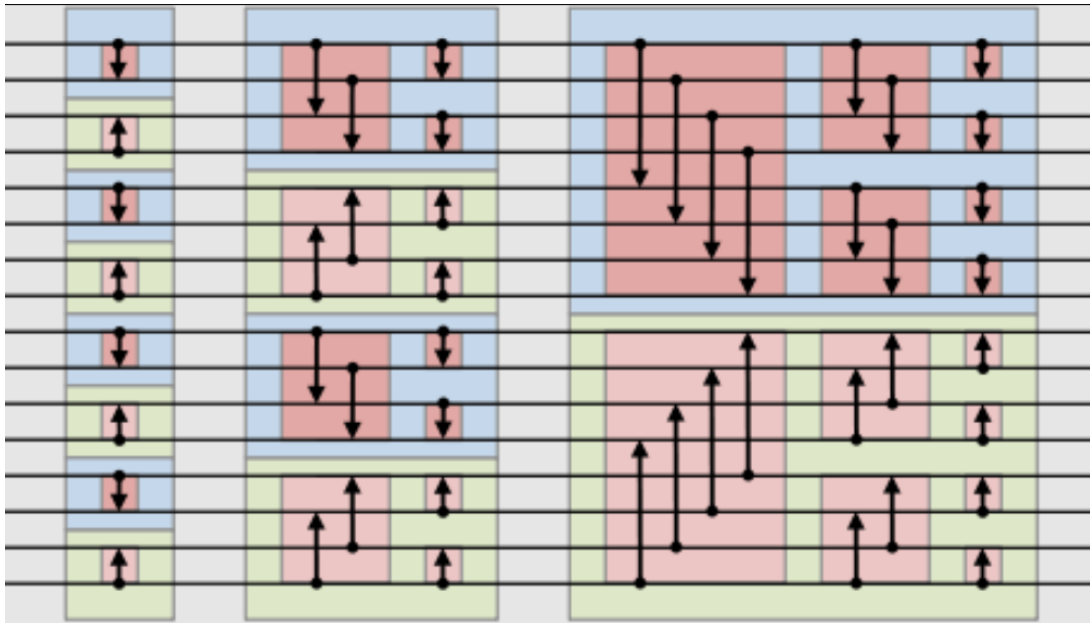


Figura 4 – Bitonic Sort para 8 Elementos

Fonte: Adaptado de (Xilinx Inc., 2019)

3 PROPOSTA DE IMPLEMENTAÇÃO

Este capítulo apresenta a arquitetura geral do sistema proposto para aceleração do algoritmo k-Nearest Neighbors (k-NN) em FPGA, descrevendo a organização modular e o fluxo de dados entre os componentes. A implementação foi projetada para maximizar o paralelismo e minimizar a latência, utilizando aritmética de ponto fixo e processamento em pipeline.

3.1 Visão Geral da Arquitetura

A arquitetura proposta implementa um acelerador hardware especializado para busca k-NN em vetores de alta dimensionalidade, otimizado para sistemas RAG que requerem recuperação eficiente de contexto. O sistema opera processando múltiplas operações em paralelo sobre vetores de embeddings, calculando suas similaridades com um vetor de consulta de referência e identificando os k vetores mais próximos. A implementação explora as características inerentes do k-NN—paralelismo massivo, acesso sequencial à memória e operações aritméticas regulares—para maximizar o desempenho em FPGA.

O processamento é realizado através de uma arquitetura pipeline composta por quatro módulos principais que operam de forma sequencial e coordenada. O fluxo completo inicia quando os vetores de embeddings chegam ao sistema no formato IEEE 754 float32, que é o padrão utilizado por modelos de linguagem e frameworks de embeddings. Cada vetor de 1024 dimensões atravessa então os seguintes estágios:

Módulo 1 - Conversor Float para Ponto Fixo: Recebe os vetores de entrada em formato float32 (1 bit de sinal, 8 bits de expoente, 23 bits de mantissa) e realiza a conversão para o formato de ponto fixo UQ0.16 (0 bits para parte inteira, 16 bits para parte fracionária). Esta conversão é crítica pois permite que todas as operações subsequentes utilizem apenas aritmética inteira, muito mais eficiente em FPGA. O módulo processa as 1024 dimensões de cada vetor sequencialmente, extraindo os campos do float32 e reconstruindo o valor na representação de ponto fixo com 16 bits totais.

Módulo 2 - Referência: Armazena o vetor de referência em memória interna organizada em 8 bancos de RAM de porta dupla. Durante os primeiros 64 ciclos a partir do primeiro valor ser liberado do pipeline do módulo 1, recebe os valores do vetor de referência já quantizados (formato UQ0.16) e os distribui pelos bancos de memória através de um sistema de controle baseado em contadores. Após a fase de escrita, o módulo opera em modo de leitura cíclica, fornecendo simultaneamente 16 valores do vetor de referência por ciclo para o módulo de cálculo de produto escalar. A arquitetura de memória dual-port

permite que o mesmo dado seja acessado paralelamente para múltiplas operações, e um sinal de fim de leitura é gerado ao completar os 64 ciclos, sincronizando o pipeline com os demais módulos do sistema.

Módulo 3 - Calculador de Produto Escalar: Recebe os vetores convertidos para ponto fixo e executa o cálculo de similaridade através do produto escalar entre o vetor de entrada e o vetor de referência fornecido pelo Módulo 2. Este módulo implementa a operação $\sum_{i=0}^{1023} (v_{entrada}[i] \times v_{referencia}[i])$ utilizando 16 multiplicadores em paralelo seguidos de uma árvore de redução combinacional e um acumulador interno. O processamento ocorre em 64 ciclos, com 16 multiplicações por ciclo, e o resultado é um único valor escalar UQ10.16 (10 bits para a parte inteira, 16 bits para a parte fracionária) que representa a similaridade entre os dois vetores—quanto maior o valor, mais próximos semanticamente estão os vetores.

Módulo 4 - Ordenador Bitônico: Recebe os valores de similaridade finais (formato UQ10.16) de cada vetor processado pelo módulo de produto escalar. Este módulo armazena os 8 scores em memória interna junto com seus índices correspondentes, e então executa uma rede de ordenação bitônica que realiza a ordenação completa em 6 ciclos de clock. A ordenação utiliza uma sequência pré-determinada de comparações e direções armazenada em ROMs, permitindo múltiplas operações de *compare-and-swap* em paralelo. A estrutura bitônica garante que, ao final dos 6 estágios, os scores estejam completamente ordenados de forma crescente, possibilitando a identificação imediata dos k vizinhos mais próximos através da leitura das primeiras k posições da memória ordenada relativas aos índices.

O diagrama da Figura 5 ilustra esta organização completa do sistema, mostrando o fluxo de dados entre os quatro módulos, as transformações de formato aplicadas em cada estágio e as estruturas de memória intermediária utilizadas. Esta arquitetura pipeline permite que diferentes vetores estejam em diferentes estágios de processamento simultaneamente, maximizando a utilização dos recursos da FPGA e minimizando a latência total do sistema.

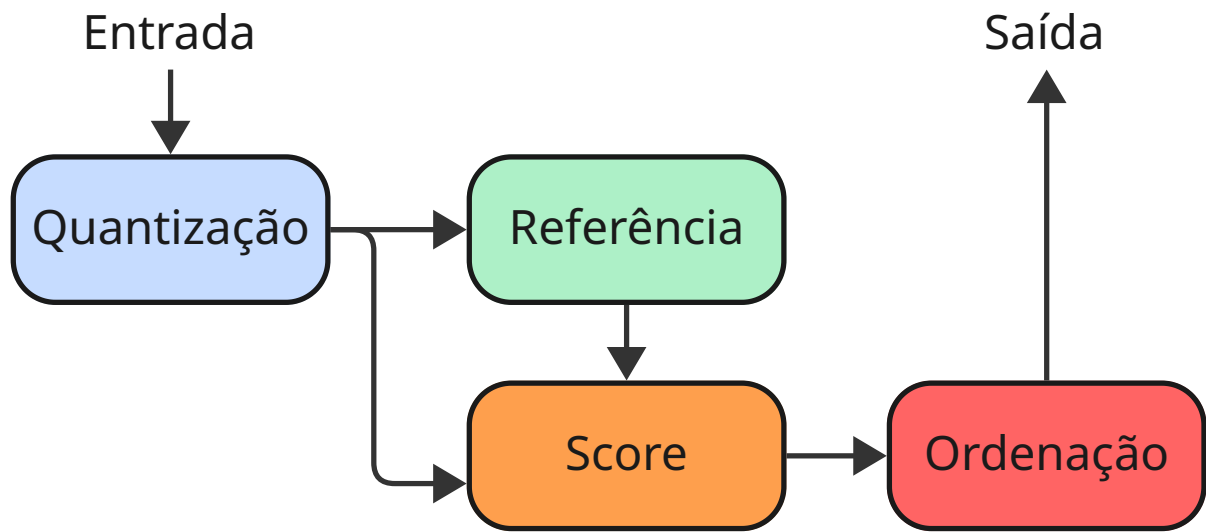


Figura 5 – Arquitetura geral do sistema de busca k-NN em FPGA

3.1.1 Escopo de Processamento

A implementação atual do sistema foi projetada e validada para processar **8 vetores de entrada** contra **1 vetor de referência**. Esta configuração foi escolhida estrategicamente como **prova de conceito funcional**, permitindo:

- **Validação do funcionamento:** Demonstrar a viabilidade técnica da aceleração hardware do algoritmo k-NN em alta dimensionalidade (1024 dimensões), confirmando que todos os módulos operam corretamente de forma integrada;
- **Análise detalhada de recursos:** Caracterizar com precisão o consumo de recursos lógicos (LUTs, flip-flops, DSP48E1, BRAMs) e temporização (frequência máxima, latência, slack) em uma implementação de tamanho controlado;
- **Identificação de gargalos:** Determinar quais recursos limitam a escalabilidade do sistema (DSP48E1, BRAMs, IOBs ou lógica combinacional);
- **Projeção de versões expandidas:** Os resultados obtidos (Capítulo 4) fornecem métricas concretas para estimar a viabilidade de configurações com maior número de vetores.

3.2 Dados de Entrada

3.3 Tipo Numérico e Dimensionalidade

Os vetores utilizados nos bancos de dados vetoriais para RAG são de alta dimensionalidade, geralmente compostos por valores em ponto flutuante simples (float32),

quantizados em inteiros ou *brain floating* (Intel Corporation, 2018). Para as entradas de dados deste trabalho, o tipo mais comum e direto foi escolhido, o float32, para que o sistema seja facilmente acoplado e utilizado sem a necessidade de conversões ou quantizações externas.

3.3.1 Análise Estatística dos Dados

Com o objetivo de reduzir a latência total do sistema e a área de ocupação para que o algoritmo possa escalar, o primeiro módulo realiza uma conversão de float32 para ponto fixo UQ0.16. Esta conversão é possível com baixa perda de precisão porque os dados em bancos de dados vetoriais para RAG são, em sua grande maioria, valores no intervalo $[-1, 1]$, pois passam por Normalização L2.

Foi realizada uma análise estatística com bibliotecas para ciência de dados na linguagem Python (Python Software Foundation, 2024): NumPy (NumPy Developers, 2024), Pandas (TEAM, 2020) e PyArrow (Apache Software Foundation, 2023) sobre o dataset *wikipedia-22-12-en-embeddings* da Cohere (Cohere AI, 2023a). O código completo da análise estatística pode ser consultado no Apêndice B, que resultou nos seguintes dados:

- 94,05% dos números estão no intervalo $[-0,7, 0,7]$;
- 98,84% dos números estão no intervalo $[-1, 1]$;
- 99,22% dos números estão no intervalo $[-2, 2]$.
- Erro de quantização: $7,63 \times 10^{-6}$

Optou-se, por este motivo, normalizar os valores no intervalo $[0, 1[$ a partir de valores no intervalo $[-2, 2]$, saturando os valores que ultrapassem esse limite (menos de 1% dos dados). Apesar da Normalização L2, erros de precisão em cálculos consecutivos originam valores fora do intervalo $[-1, 1]$.

Para a dimensionalidade da entrada foi adotado o valor de 1024, compatível com bases de dados modernas, como o *wikipedia-22-12-en-embeddings*.

3.4 Módulo de Quantização

Este módulo é responsável pela conversão eficiente de valores em ponto flutuante IEEE 754 (float32) para o formato de ponto fixo UQ0.16, preservando a precisão necessária para o cálculo de similaridade vetorial.

3.4.1 Design da Quantização

O design da quantização está representado nas Figuras 6 e 7.

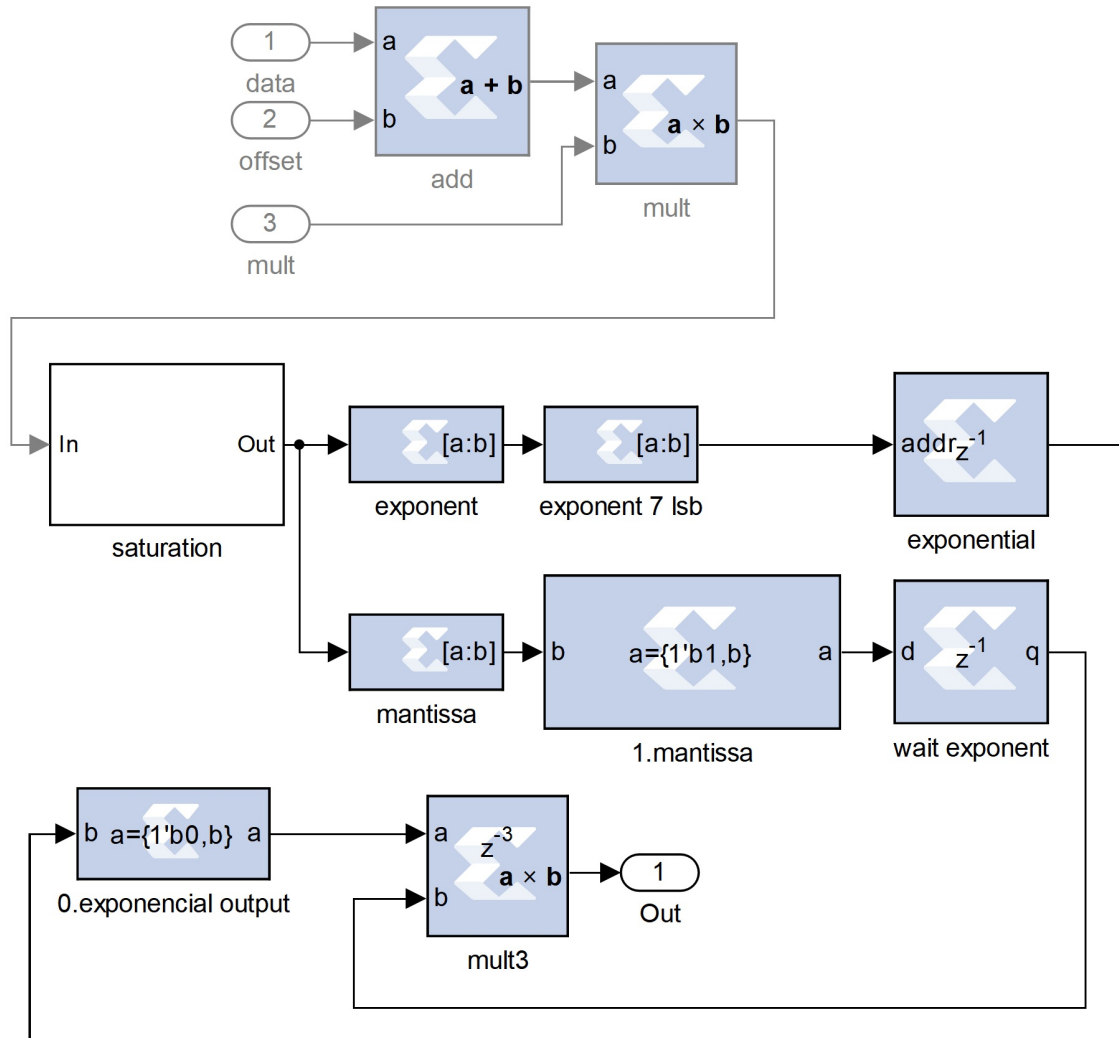


Figura 6 – Módulo de Quantização

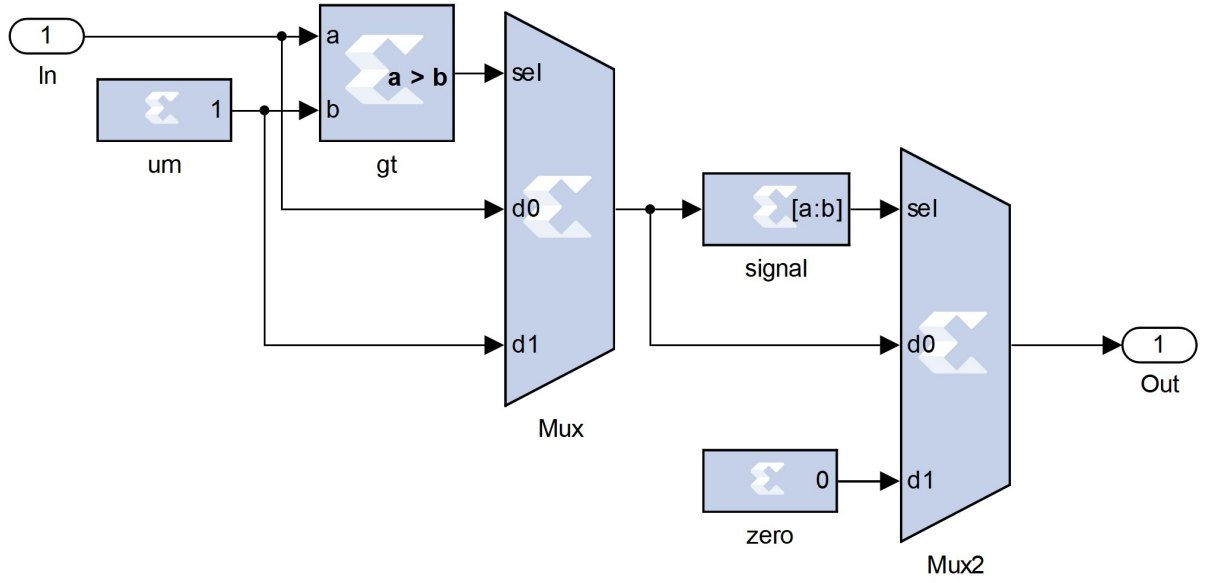


Figura 7 – Módulo de Quantização – Saturação

A primeira etapa do processo é mapear os valores para o intervalo $[0, 1]$, conforme a Equação 3.1.

$$V_{\text{norm}} = \text{sat}((V_{\text{real}} + \text{offset}) \times \text{mult}) \quad (3.1)$$

onde:

- V_{real} é o valor original em ponto flutuante;
- offset é o deslocamento aplicado ($\text{offset} = 2$);
- mult é o fator de escala ($\text{mult} = 0,25$);
- $\text{sat}(\cdot)$ é a função de saturação que limita os valores ao intervalo $[0, 1]$.

O intervalo dos valores em cada etapa da normalização é:

$$[-2, 2] + 2 = [0, 4] \quad (3.2)$$

$$[0, 4] \times 0,25 = [0, 1] \quad (3.3)$$

$$\text{sat}(x) = \begin{cases} 0 & \text{se } x < 0 \\ x & \text{se } 0 \leq x < 1 \\ 1 & \text{se } x \geq 1 \end{cases} \quad (3.4)$$

A partir do valor normalizado, o barramento relativo ao número é dividido em mantissa e expoente; o sinal não é utilizado, pois é sempre zero. Baseado na Equação

2.1 e considerando que o valor está normalizado, o expoente é necessariamente um valor no intervalo $[0, 127]$, pois $E - 127$ precisa estar no intervalo $[-127, 0]$, uma vez que a mantissa é um valor maior ou igual a um. Pelo fato do valor máximo ser 127, o bit MSB do expoente não é utilizado.

Os 7 *least significant bits* (LSB) do expoente são usados como endereço de uma *Read-Only Memory* (ROM); a saída da ROM é um valor em ponto fixo UQ0.16. Esta técnica de utilização de LUT exige baixo consumo de recursos, baixa latência e evita processamentos pesados como os de uma exponencial.

A mantissa recebe um bit MSB de valor um e é reinterpretada como ponto fixo UQ1.23, finalizando a adição do bit implícito do padrão IEEE 754. Por fim, o valor da exponencial recebe um bit MSB zero e é multiplicado pela mantissa com o bit implícito; a saída da multiplicação é fixada em formato UQ0.16, já que o resultado da multiplicação deve ser compatível com o valor real que está no intervalo $[0, 1]$, finalizando a quantização.

3.4.2 Vantagens da Abordagem

A estratégia de quantização proposta oferece:

- **Baixa latência:** Uso de ROM para cálculo de exponencial evita operações complexas;
- **Economia de recursos:** Redução significativa de LUTs, registradores e DSPs;
- **Saturação inteligente:** Tratamento adequado de valores fora do intervalo esperado;
- **Escalabilidade:** Design modular permite processamento paralelo de múltiplos vetores.

3.5 Módulo Referência

O módulo Referência na Figura 8 é formado de um banco de 8 random-access memory (RAM) de dois acessos simultâneos e uma unidade de controle. Os primeiros 64 clocks do sistema são para recebimento dos dados do vetor de referência, que são escritos nestas memórias a partir do quarto clock, que é o momento que o *pipelining* do sistema permite que o módulo Referência comece a receber os primeiros valores quantizados.

Quando o sistema inicia, um *bit* habilitador permanece em nível lógico '1', o que inicia os contadores do controle da referência. Estes contadores controlam os endereços das memórias durante os ciclos de escrita e leitura. Eles podem ser resetados externamente para um novo processamento do algoritmo.

O controle faz uma verificação de clock entre 4 e 67 para a escrita e em seguida fica em *loop* de contagem cíclica para a leitura das memórias. Durante a fase de leitura

sempre que o valor de endereço B das memórias chega no valor máximo, é emitido um *bit* de fim de leitura pelo o controlador do módulo de Score, que tem influência no controle do acumulador do módulo Score e no endereçamento da memória de entrada do módulo Ordenação.

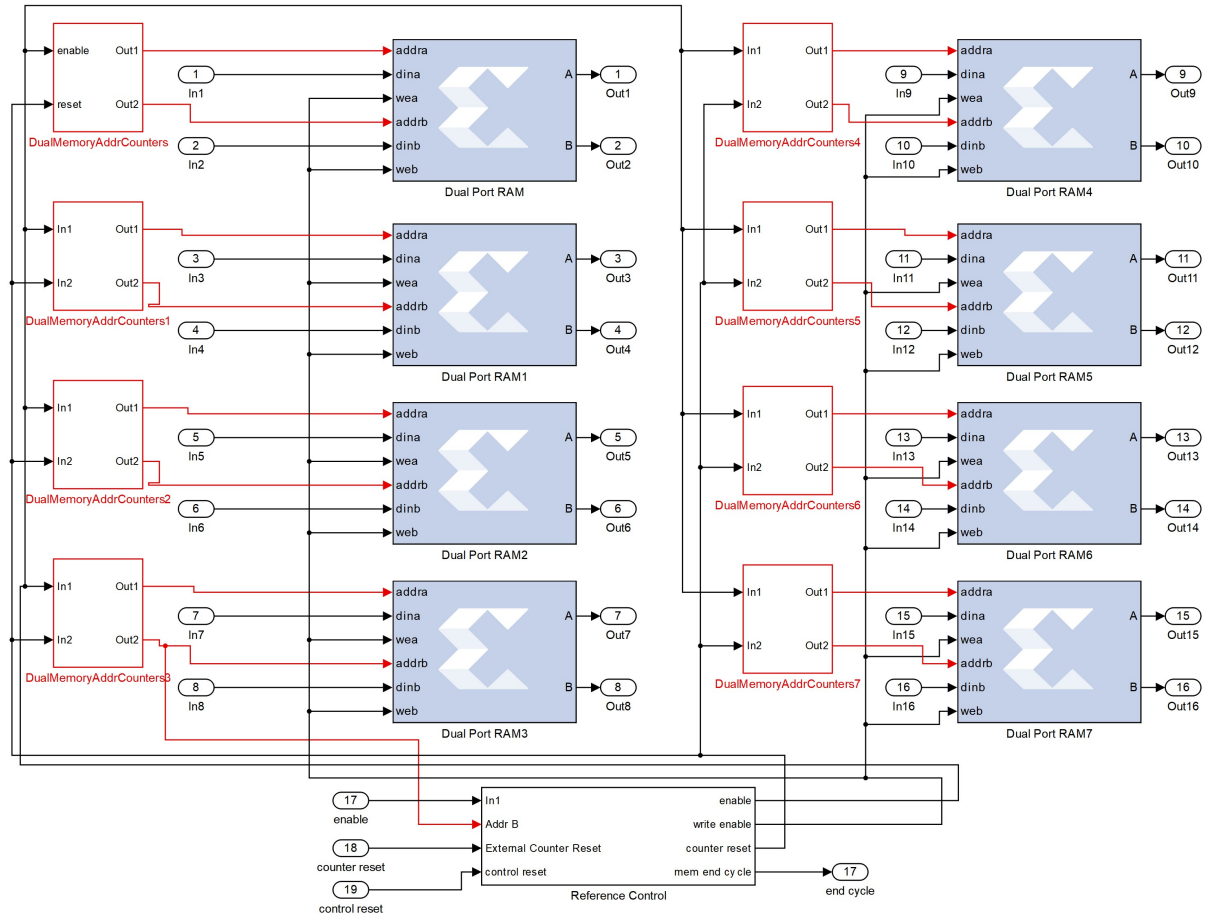


Figura 8 – Módulo Referência

3.6 Módulo Score

O módulo Score é responsável pelo cálculo do produto interno entre segmentos dos vetores de entrada e o vetor de referência armazenado em memória. Este módulo realiza 16 multiplicações paralelas seguidas de uma redução em soma através de uma árvore combinacional de 4 estágios, conforme ilustrado na Figura 9.

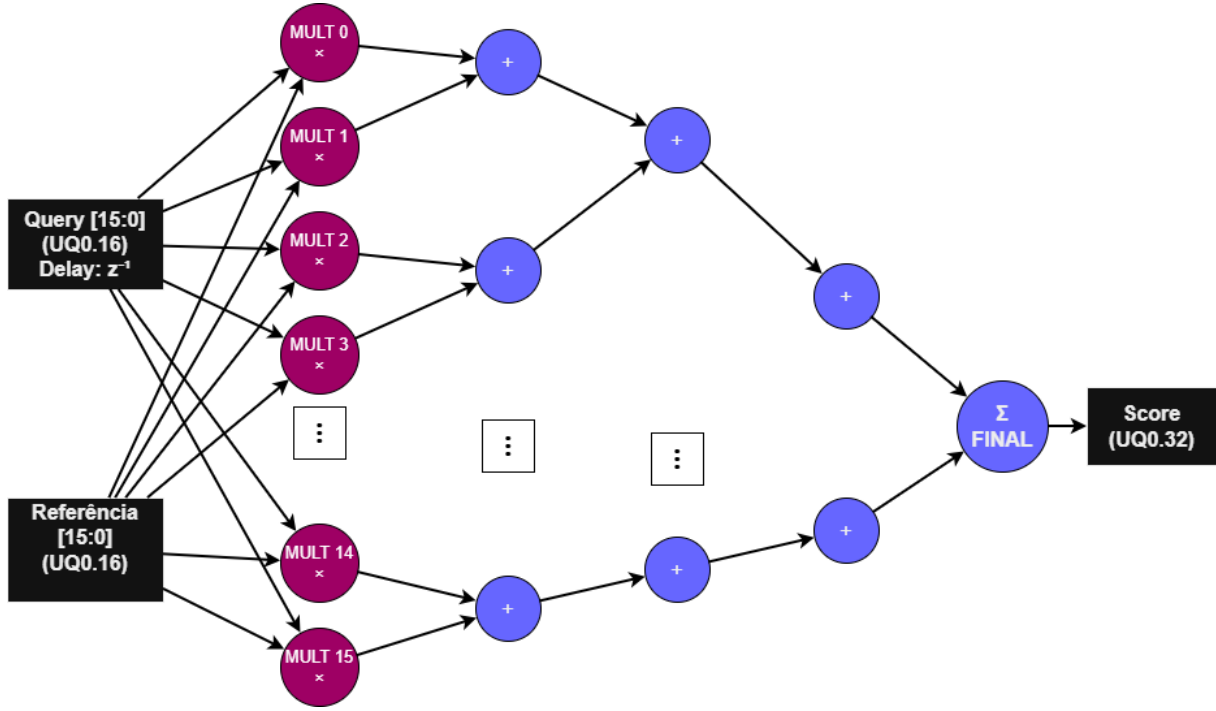


Figura 9 – Módulo Score – Diagrama de blocos

3.6.0.1 Processamento por Ciclo

A cada ciclo k (onde $k \in \{0, 1, \dots, 63\}$), o hardware executa:

$$\text{Score}_{\text{parcial}}[k] = \sum_{j=0}^{15} x_{16k+j} \times r_{16k+j} \quad (3.5)$$

Este cálculo é realizado em três etapas:

1. **Multiplicações paralelas:** 16 blocos DSP48E1 computam simultaneamente os produtos $x_{16k+j} \times r_{16k+j}$ para $j = 0, 1, \dots, 15$;
2. **Redução em árvore:** Os 16 produtos são somados através de uma árvore de redução;
3. **Acumulação:** O resultado parcial é somado ao acumulador interno, que mantém a soma cumulativa de todos os blocos já processados.

3.6.0.2 Modo de Operação em Streaming

O sistema opera em modo de *streaming*, o que significa que:

- **Processamento sequencial:** Os vetores de entrada são processados um de cada vez, em ordem, contra o vetor de referência;
- **Pipeline contínuo:** Enquanto o bloco k do vetor n está sendo processado, o bloco $k - 3$ (considerando latência de pipeline) do mesmo vetor já foi multiplicado e está na árvore de redução;
- **Throughput constante:** Após o pipeline estar cheio (3 ciclos iniciais), um score parcial é produzido a cada ciclo de clock;
- **Intercalação de vetores:** Novo vetor pode iniciar processamento imediatamente após o anterior completar seus 64 ciclos, sem ciclos ociosos entre vetores diferentes.

Esta estratégia de streaming garante alta eficiência de utilização dos recursos, mantendo os blocos DSP48E1 constantemente ocupados durante toda a fase de processamento de múltiplos vetores.

3.6.1 Sincronização com o Módulo Referência

Uma característica crítica do design é o atraso de z^{-1} (delay de -1 ciclo de clock) aplicado aos dados provenientes do módulo Referência na entrada do operando B dos multiplicadores. Este atraso é necessário para alinhar temporalmente os dados do vetor de referência com os dados do vetor de entrada, compensando a latência de leitura das memórias RAM de porta dupla presentes no módulo Referência.

Quando o módulo Referência recebe um endereço de leitura no ciclo n , os dados correspondentes ficam disponíveis nas portas de saída apenas no ciclo $n + 1$. O delay aplicado garante que:

- No ciclo n : O endereço é enviado para as RAMs do módulo Referência;
- No ciclo $n + 1$: Os dados são lidos das RAMs e propagados para os multiplicadores;
- No ciclo $n + 1$: Os dados do vetor de entrada (sem delay) chegam simultaneamente aos multiplicadores, perfeitamente sincronizados com os dados de referência.

Sem este alinhamento temporal, haveria um *mismatch* entre os índices dos elementos sendo multiplicados, resultando em cálculos incorretos do produto interno.

3.6.2 Pipeline e Latência do Módulo

O módulo Score possui uma latência total de 3 ciclos de clock, determinada pelos seguintes estágios:

1. **Ciclo 1:** Registro de entrada dos operandos nos multiplicadores;
2. **Ciclo 2:** Execução da multiplicação nos blocos DSP48E1;
3. **Ciclo 3:** Registro de saída dos produtos.

A redução em soma (somatório dos 16 produtos) é implementada como uma árvore combinacional balanceada de 4 estágios:

- **Estágio 1:** 8 somadores ($16 \rightarrow 8$ valores)
- **Estágio 2:** 4 somadores ($8 \rightarrow 4$ valores)
- **Estágio 3:** 2 somadores ($4 \rightarrow 2$ valores)
- **Estágio 4:** 1 somador final ($2 \rightarrow 1$ valor)

A latência da árvore de redução é considerada zero ciclos de clock porque o circuito combinacional é suficientemente pequeno para que todos os sinais elétricos se estabilizem antes da borda de subida do próximo clock. O tempo de propagação total da árvore é inferior ao período do clock de operação do sistema (50 MHz, período de 20 ns), permitindo que o resultado esteja disponível no mesmo ciclo em que os produtos são gerados.

Esta arquitetura combinacional para a redução oferece vantagens significativas:

- **Throughput máximo:** Um score parcial é gerado a cada ciclo de clock;
- **Baixa latência:** Apenas 3 ciclos de atraso do pipeline dos multiplicadores;
- **Economia de recursos:** Não requer registradores intermediários para armazenar resultados parciais da soma;
- **Determinismo:** Latência fixa e previsível para todos os cálculos.

3.6.3 Formato de Dados e Evolução da Precisão

Os dados de entrada nos multiplicadores estão no formato UQ0.16 (16 bits sem sinal, ponto fixo fracionário). A multiplicação de dois valores UQ0.16 resulta em um produto de 32 bits que, após truncamento, retorna ao formato UQ0.16. A árvore de soma precisa acomodar o crescimento da largura de bits devido às adições sucessivas, mantendo sempre 16 bits para a parte fracionária:

- **Saída dos multiplicadores:** UQ0.16 (16 bits)
- **Estágio 1 (soma de 2 valores):** UQ2.16 (18 bits)
- **Estágio 2 (soma de 4 valores):** UQ3.16 (19 bits)
- **Estágio 3 (soma de 8 valores):** UQ4.16 (20 bits)
- **Estágio 4 (soma de 16 valores):** UQ6.16 (22 bits)

O score parcial de saída mantém o formato UQ6.16 com 22 bits totais (6 bits inteiros + 16 bits fracionários), suficiente para representar a soma de 16 produtos.

3.6.4 Acumulação de Scores Parciais e Sincronização com o Módulo Ordenação

O módulo Score incorpora um acumulador interno que soma os 64 valores parciais (Equação 3.5) gerados durante o processamento completo de um vetor de 1024 dimensões. Este acumulador opera da seguinte forma:

1. **Inicialização:** No início do processamento de um novo vetor, o acumulador é resetado para zero;
2. **Acumulação:** A cada ciclo de clock, durante os 64 ciclos de processamento, o score parcial (formato UQ5.16) é somado ao valor acumulado;
3. **Crescimento de bits:** O acumulador precisa comportar o valor máximo teórico de aproximadamente 1024, considerando que cada dimensão pode ter valor próximo a 1. No formato UQ0.16, o maior valor representável é:

$$\text{Valor}_{\text{máx}}^{\text{UQ0.16}} = \frac{2^{16} - 1}{2^{16}} = \frac{65535}{65536} \approx 0,999984741 \quad (3.6)$$

Somando-se 1024 valores máximos UQ0.16, o resultado tende a:

$$\text{Score}_{\text{máx}} = 1024 \times 0,999984741 \approx 1023,984 \approx 1024 \quad (3.7)$$

Para acomodar com segurança valores até quase 1024, são necessários 10 bits inteiros ($2^{10} = 1024$) com os 16 bits fracionários padrão da proposta; o formato resultante do acumulador é UQ10.16 (26 bits totais: 10 bits inteiros + 16 bits fracionários):

Este dimensionamento garante que nenhum overflow ocorra mesmo no caso extremo (improvável na prática) em que todas as 1024 dimensões contenham o valor máximo representável em UQ0.16 em ambos vetores.

4. **Finalização:** Quando o módulo Referência emite o bit de fim de leitura (após completar os 64 ciclos), o valor final acumulado representa o produto interno completo conforme a Equação 2.6.

O bit de fim de leitura gerado pelo módulo Referência atua como um sinal de sincronização crítico no pipeline do sistema. Este sinal é emitido quando o módulo Referência completa a leitura do 64º ciclo do vetor de referência. Para garantir o correto alinhamento temporal com o término do processamento do 64º score parcial, o bit de fim de leitura é atrasado em 5 ciclos de clock.

Este atraso de 5 ciclos compensa:

- **Latência do pipeline dos multiplicadores:** 3 ciclos;
- **Latência da árvore de redução combinacional:** 0 ciclos (circuito combinacional dentro do mesmo ciclo de clock);
- **Registrador de acumulação:** 1 ciclo para armazenar o resultado da soma no acumulador;
- **Finalização da última soma no acumulador:** 1 ciclo.

Quando o bit de fim de leitura atrasado chega ao módulo Score, desencadeia as seguintes ações:

- **Finalização do cálculo:** Confirma que todos os 64 scores parciais foram processados e acumulados;
- **Disponibilização do resultado:** O valor final do acumulador (formato UQ10.16) representa o produto interno completo;
- **Notificação implícita ao módulo Ordenação:** O bit de fim de leitura, após passar pelo módulo Score, propaga-se para o módulo Ordenação com o atraso adequado;
- **Gravação na memória de ordenação:** O módulo Ordenação armazena o score final em sua memória interna, associado ao índice do vetor processado;
- **Reset do acumulador:** O acumulador é zerado, preparando o sistema para o processamento do próximo vetor de entrada.

Este mecanismo de sincronização garante que:

1. **Integridade dos dados:** Cada score final corresponde exatamente a um vetor completo de 1024 dimensões;

2. **Sincronização precisa:** O atraso de 5 ciclos garante que o bit de fim de leitura chegue exatamente quando o último score parcial foi acumulado;
3. **Ordenação sequencial:** Os scores são gravados na ordem de processamento dos vetores de entrada;
4. **Pipeline contínuo:** Enquanto um score está sendo finalizado e gravado no módulo Ordenação, o próximo vetor já pode iniciar seu processamento no módulo Score;
5. **Controle de fluxo distribuído:** O bit de fim de leitura atua como handshake entre os módulos Referência, Score e Ordenação, propagando-se pelo pipeline com os atrasos apropriados.

A Figura 10 ilustra o diagrama de temporização do processo de acumulação e notificação:

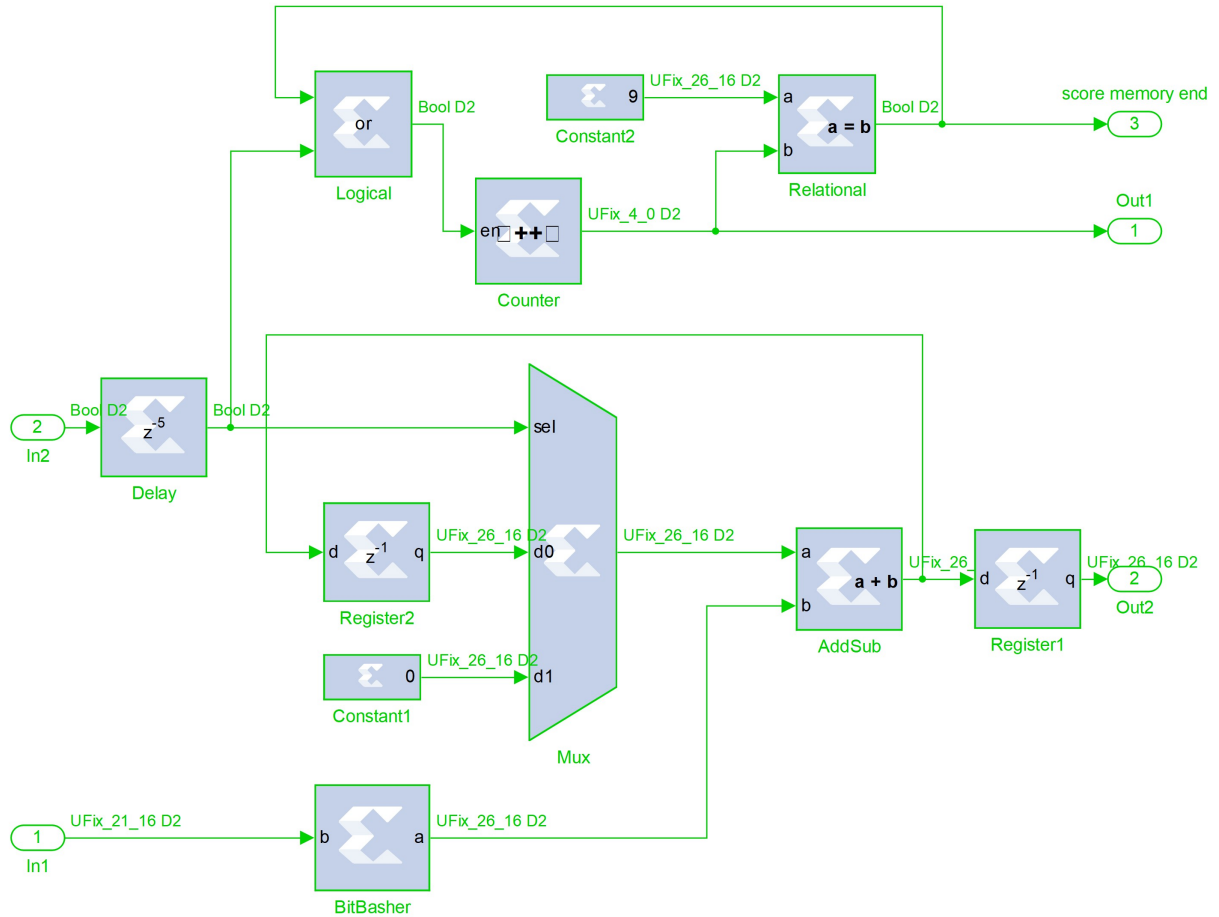


Figura 10 – Diagrama de temporização – Acumulação e notificação de score

3.6.5 Desempenho e Escalabilidade

O módulo Score processa 16 multiplicações e 15 adições por ciclo de clock. Para um vetor completo de 1024 dimensões:

- **Ciclos necessários:** 64 ciclos (1024/16)
- **Latência inicial (pipeline):** 3 ciclos
- **Latência total por vetor:** 67 ciclos (3 + 64)
- **Taxa de processamento:** 16 operações por ciclo
- **Throughput:** 1 produto interno completo a cada 64 ciclos (após pipeline cheio)
- **Formato de saída final:** UQ10.16 (26 bits)

3.6.6 Teste em Ambiente de Simulação

Para validar o funcionamento correto do módulo Score, foi realizada uma simulação completa no Simulink utilizando vetores de entrada uniformemente aleatórios no intervalo $[-2, 2]$. Os valores simulados apresentados na Figura 11 correspondem às saídas da árvore de redução de soma do módulo Score, representando os scores parciais gerados a cada ciclo de clock durante o processamento de um vetor de 1024 dimensões.

A cada ciclo, a árvore de redução calcula o somatório dos 16 produtos, conforme descrito na Equação 3.5, produzindo um score parcial que é então acumulado internamente. O gráfico mostra a evolução desses scores parciais ao longo dos 64 ciclos necessários para processar cada vetor, permitindo verificar se o comportamento do hardware implementado corresponde aos valores teóricos calculados através de produto interno no Matlab, utilizando os mesmos dados de entrada da simulação.

A coleta de dados da simulação foi realizada utilizando blocos *Gateway Out* do System Generator conectados a blocos *To Workspace* do Simulink, que armazenam os valores de saída da árvore de redução em variáveis do MATLAB para análise posterior. Esta configuração permite capturar os valores ciclo a ciclo sem interferir no funcionamento do circuito.

A Figura 11 demonstra o alinhamento perfeito entre os valores teóricos e simulados para um dos vetores de entrada. Importante ressaltar que este mesmo procedimento foi repetido para todos os 8 vetores processados pelo sistema, e em todos os casos os gráficos resultantes apresentaram alinhamento perfeito entre os valores teóricos e simulados, confirmando a corretude da implementação do módulo Score em toda sua operação. Esta consistência valida tanto a aritmética de ponto fixo quanto a lógica de acumulação implementada no hardware.

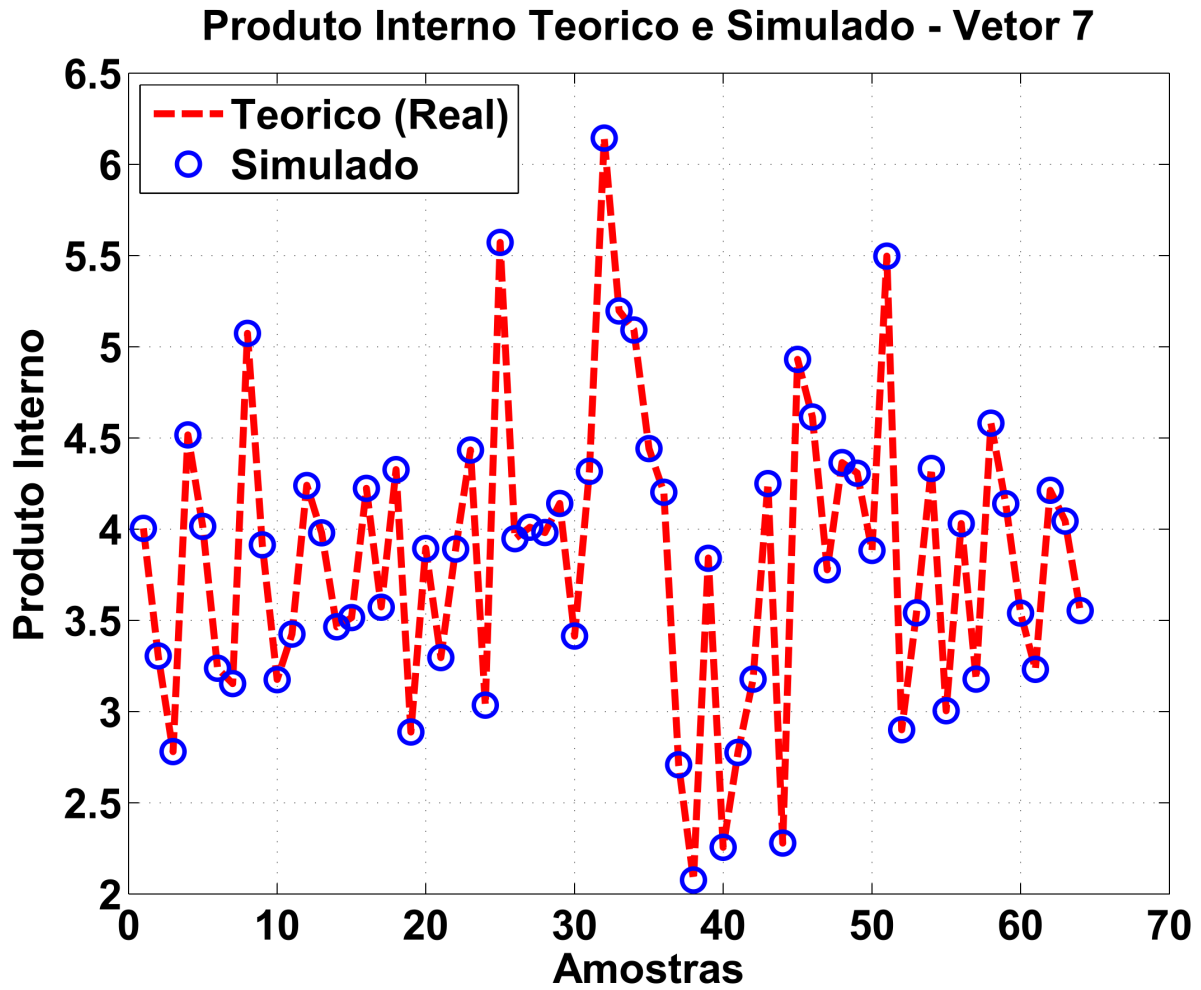


Figura 11 – Simulação do Módulo Score – Valores Teóricos vs Simulados

3.7 Módulo de Ordenação

O módulo de ordenação é responsável por organizar os k menores scores calculados pelo módulo Score, identificando os vizinhos mais próximos ao vetor de entrada. Algoritmos de ordenação tradicionais, como bubble sort e insertion sort, apresentam complexidade $O(n^2)$ inadequada para implementação em hardware de alto desempenho. Por outro lado, algoritmos baseados em redes de ordenação, como Bitonic Sort e Odd-Even Merge Sort, oferecem paralelismo natural e estrutura regular ideal para FPGAs.

Conforme apresentado na seção de Trabalhos Relacionados (Capítulo 1), (ABDEL-RASOUL; SHABAN; ABDEL-KADER, 2021) demonstrou que o Bitonic Sort é preferido por projetistas de hardware devido à sua estrutura altamente regular, que facilita síntese e roteamento em FPGA, embora apresente área ligeiramente maior que o Odd-Even Merge Sort. Com base nesta análise, o presente trabalho adota o Bitonic Sort para o módulo de ordenação devido às seguintes características:

- **Estrutura regular e previsível:** Facilita implementação em hardware e otimização de temporização
- **Paralelismo natural:** Permite execução simultânea de múltiplas comparações em cada estágio
- **Pipeline eficiente:** Estrutura em estágios permite alta taxa de throughput com latência determinística

3.7.1 Arquitetura do Módulo

O módulo de ordenação é composto por quatro componentes principais que operam de forma coordenada:

1. **Módulo de Controle:** Gerencia a sequência de comparações e sincroniza os demais componentes através de contadores e máquinas de estado, conforme ilustrado na Figura 12;

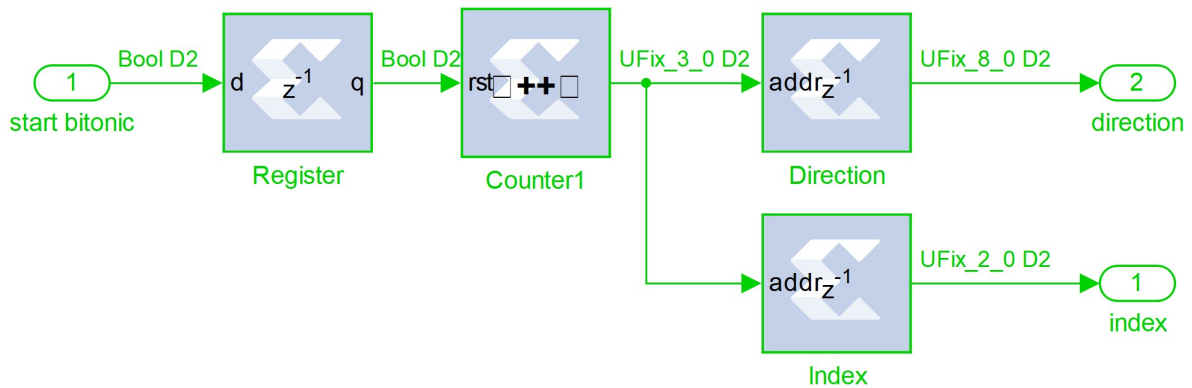


Figura 12 – Módulo de Controle da Ordenação

2. **ROMs de Configuração:** Duas memórias ROM que armazenam, para cada ciclo de clock:
 - **ROM de Direção:** Define se a comparação deve ser crescente (menor para maior) ou decrescente (maior para menor);
 - **ROM de Seleção:** Indica quais pares de elementos devem ser comparados em cada estágio da rede de ordenação;
3. **Memória de Dados:** Armazena simultaneamente os scores (formato UQ10.16) e os índices correspondentes dos vetores, conforme mostrado na Figura 13. Esta memória possui capacidade para 8 entradas, cada uma contendo:
 - Score: 26 bits (UQ10.16)

- Índice: $\lceil \log_2(8) \rceil = 3$ bits
- Total por entrada: 29 bits

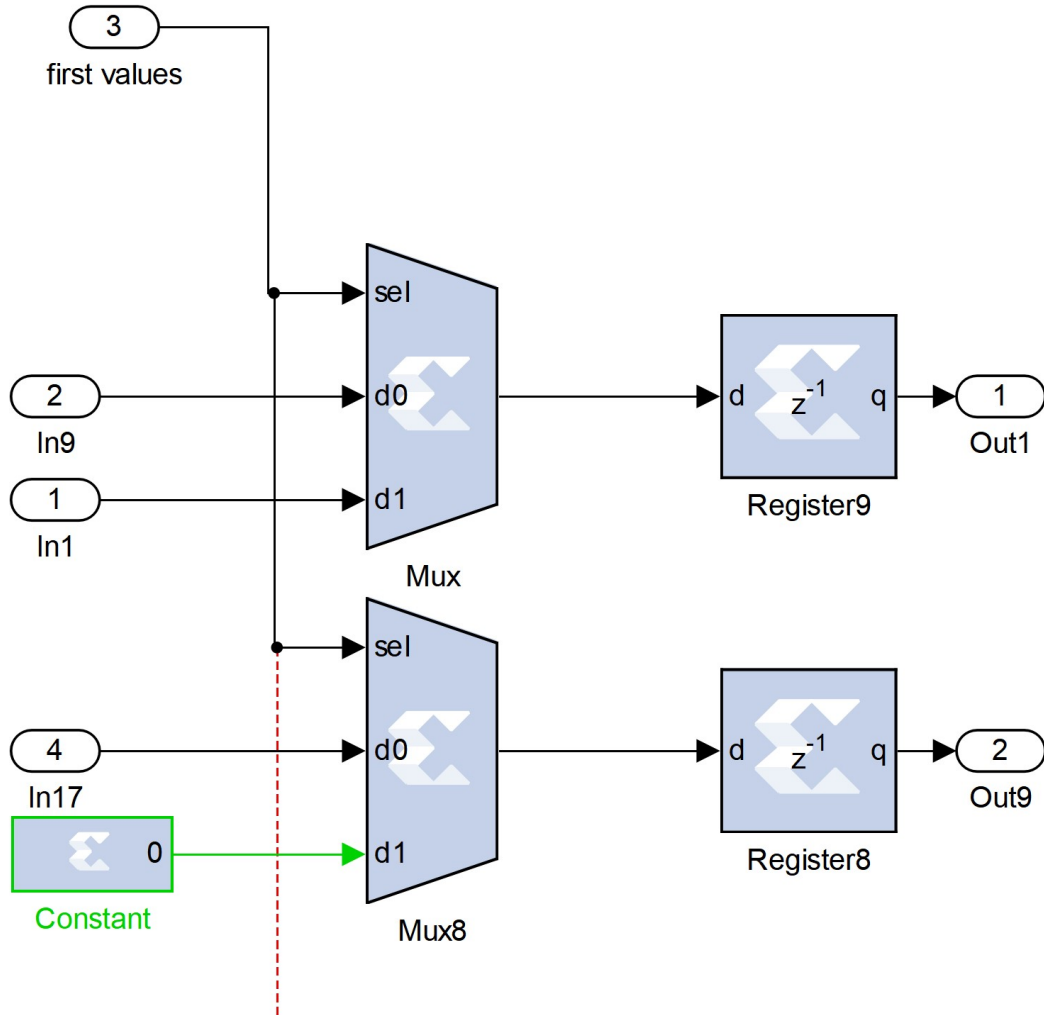


Figura 13 – Memória de Dados da Ordenação

4. **Unidade de Swap:** Circuito combinacional que realiza as operações de comparação e troca (*compare-and-swap*), detalhado na Figura 14. Dada a direção de comparação e os elementos selecionados por multiplexadores, esta unidade:

- Compara os dois scores selecionados;
- Determina se é necessária uma troca baseada na direção especificada;
- Calcula os novos valores (score e índice) a serem escritos na memória no próximo ciclo de clock;
- Opera simultaneamente nos scores e nos índices associados, garantindo que a correspondência entre score e vetor seja mantida.

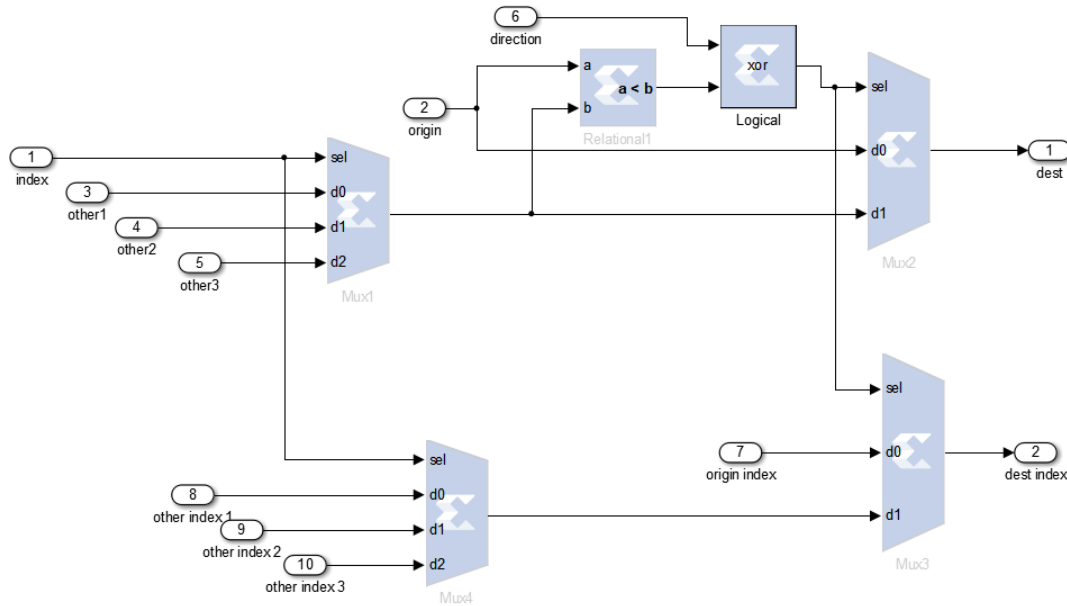


Figura 14 – Unidade de Swap (Compare-and-Swap)

3.7.2 Funcionamento Ciclo a Ciclo

O processo de ordenação opera da seguinte forma:

1. **Recepção de dados (Ciclos 0-7):** À medida que o módulo Score finaliza o cálculo de cada produto interno e o bit de fim de leitura é recebido, o score e seu índice correspondente são escritos sequencialmente na memória de dados;
2. **Inicialização da ordenação (Ciclo 8):** Quando todos os 8 scores foram recebidos, o módulo de controle inicia a sequência de ordenação;
3. **Estágios de comparação (Ciclos 8-13):** Durante 6 ciclos consecutivos, o módulo executa:
 - O contador do módulo de controle incrementa a cada ciclo;
 - A ROM de Direção é endereçada pelo contador, fornecendo o bit de direção para o estágio atual;
 - A ROM de Seleção é endereçada pelo contador, fornecendo os índices do mux com o valor do outro elemento a ser comparado;
 - Os multiplexadores selecionam os elementos indicados pela ROM de Seleção;
 - A unidade de Swap compara os scores selecionados e, se necessário, troca tanto os scores quanto os índices;
 - Na borda do próximo clock, os valores atualizados são escritos de volta na memória;

4. **Finalização (Ciclo 14):** Após os 6 estágios, a memória contém:

- Os 8 scores ordenados;
- Os índices correspondentes, permitindo identificar qual vetor de entrada gerou cada score;

5. **Leitura dos resultados:** Os scores ordenados e seus índices podem ser lidos da memória para determinação dos k vizinhos mais próximos.

3.7.3 Operação de Swap

A unidade de Swap implementa a operação fundamental de *compare-and-swap*. A comparação entre scores UQ10.16 é realizada através de um bloco comparador de 26 bits, que gera um sinal lógico indicando se o valor da posição de memória é menor que o selecionado pelo mux. A troca simultânea de scores e índices é implementada através de multiplexadores controlados pelo resultado da comparação e pelo bit de direção, conforme mostrado na Figura 14.

3.7.4 Conteúdo das ROMs

As ROMs armazenam a sequência pré-calculada de comparações para a rede bitônica de 8 elementos. Duas ROMs são utilizadas:

3.7.4.1 ROM de Direção

A ROM de Direção contém valores de 8 bits onde cada bit representa a direção de comparação para um par de elementos. O bit em '0' indica comparação crescente (menor para maior) e o bit em '1' indica comparação decrescente (maior para menor). A Tabela 4 mostra o conteúdo da ROM:

Endereço	Binário	Decimal	Padrão de Direção
0	01100110	102	↑↓↑↑↑↓↑
1	11000011	195	↓↓↑↑↑↓
2	10100101	165	↓↑↓↑↑↓
3	11110000	240	↓↓↓↑↑↑
4	11001100	204	↓↑↑↓↑↑
5	10101010	170	↓↑↓↑↓↑

Tabela 4 – Conteúdo da ROM de Direção

3.7.4.2 ROM de Seleção (Bitonic Index)

A ROM de Seleção indica a seleção para o mux que seleciona os pares de elementos a serem comparados em cada estágio. A Tabela 5 mostra o conteúdo:

Endereço	Index	Pares Comparados
0	0	(0,1), (2,3), (4,5), (6,7)
1	1	(0,2), (1,3), (4,6), (5,7)
2	0	(0,1), (2,3), (4,5), (6,7)
3	2	(0,4), (1,5), (2,6), (3,7)
4	1	(0,2), (1,3), (4,6), (5,7)
5	0	(0,1), (2,3), (4,5), (6,7)

Tabela 5 – Conteúdo da ROM de Seleção (Bitonic Index)

3.7.5 Desempenho e Características

O módulo de ordenação apresenta as seguintes características de desempenho:

- **Latência fixa:** 6 ciclos de clock para ordenar 8 elementos;
- **Throughput:** Um conjunto ordenado a cada 6 + tempo de recepção de dados ciclos;
- **Escalabilidade:** Para N vetores, requer $O(\log^2 N)$ ciclos;
- **Determinismo:** Latência constante independente dos dados de entrada.

3.7.6 Vantagens da Implementação em Hardware

A ordenação em hardware oferece benefícios significativos em relação a implementações em software:

- **Paralelismo real:** Múltiplas comparações ocorrem simultaneamente no mesmo ciclo quando não há dependências de dados;
- **Latência previsível:** Sempre 6 ciclos, independente da ordem inicial dos dados;
- **Pipeline integration:** Integra-se naturalmente com os módulos Score e Referência sem necessidade de sincronização complexa.

3.8 Saída

Após a conclusão do processamento e ordenação de todos os vetores de entrada, o sistema disponibiliza como saída os índices dos k vetores mais próximos ao vetor de referência, ordenados de forma crescente conforme seus valores de similaridade. A saída consiste exclusivamente nos índices, não incluindo os valores de score calculados.

4 RESULTADOS E ANÁLISE

Este capítulo apresenta os resultados da síntese e implementação do k-NN proposto, incluindo análise de temporização, utilização de recursos e desempenho do sistema.

4.1 Análise de Temporização

4.1.1 Resultados de Temporização

O sistema foi projetado para operar a 50 MHz (período de 20 ns). A análise de temporização reportou:

- **Período mínimo atingido:** 18,598 ns
- **Frequência máxima:** 53,769 MHz
- **Slack:** 1,402 ns (margem de 7,0%)
- **Erros de temporização:** 0
- **Caminhos analisados:** 1.805.988.592

O slack positivo indica que o design atende confortavelmente à restrição de 50 MHz, garantindo operação confiável mesmo sob variações de processo, tensão e temperatura.

4.1.2 Caminho Crítico

O caminho crítico (18,403 ns) atravessa a cadeia de computação do produto interno:

1. **Origem:** DSP48E1 multiplicador 11 (0,494 ns)
2. **Árvore de redução (4 níveis):** Nível 1 (2,034 ns), Nível 2 (2,034 ns), Nível 3 (1,824 ns), Nível 4 (2,034 ns)
3. **Acumulação:** DSP48E1 acumulador (2,063 ns)
4. **Destino:** Flip-flop de controle
5. **Roteamento total:** 7,909 ns (43,0%)

O uso de blocos DSP48E1 dedicados resulta em delays de lógica previsíveis e otimizados, com delay de roteamento adequado para FPGAs de grande porte.

4.2 Utilização de Recursos

A Tabela 6 apresenta a utilização completa de recursos da FPGA:

Recurso	Utilizado	Disponível	Utilização (%)
Lógica:			
Slice Registers	1.201	301.440	0,4
Flip-Flops	1.121	–	–
Latches	0	–	–
Slice LUTs	5.580	150.720	3,7
Como lógica	5.537	–	–
Como memória (shift reg)	1	–	–
Como route-thrus	42	–	–
Occupied Slices	1.986	37.680	5,3
Control sets únicos	4	–	–
Memória:			
RAMB18E1	26	832	3,1
RAMB36E1	0	416	0,0
DSP:			
DSP48E1	130	768	16,9
I/O:			
IOBs (Bonded)	523	600	87,2
Clock:			
BUFG/BUFGCTRLs	1	32	3,1
MMCM_ADVs	0	12	0,0
Roteamento:			
Fanout médio	2,36	–	–
Sinais não roteados	0	–	–

Tabela 6 – Utilização de recursos da FPGA Virtex-6 XC6VLX240T

4.2.1 Análise dos Recursos Críticos

A análise de utilização de recursos revela o perfil de consumo do acelerador hardware k-NN e identifica os limitadores de escalabilidade do sistema.

4.2.1.1 DSP48E1 (16,9% - 130 de 768 blocos)

Os blocos DSP48E1 representam o recurso computacional mais demandado, consumindo 16,9% da capacidade disponível. Esta utilização reflete a natureza intensiva em operações aritméticas do algoritmo k-NN, onde cada produto interno de 1024 dimensões requer múltiplas multiplicações e somas em paralelo. A utilização moderada indica margem significativa para replicação do módulo Score, permitindo expansão de até $5,9\times$ (limitado pela disponibilidade de 768 blocos totais) caso o gargalo de I/O seja resolvido.

4.2.1.2 RAMB18E1 (3,1% - 26 de 832 blocos)

As Block RAMs apresentam consumo extremamente baixo (3,1%), demonstrando que a arquitetura de memória está otimizada para o conjunto de dados atual. Com 806 blocos ainda disponíveis (832 - 26). Este resultado indica que BRAMs não representam limitação para aplicações típicas de RAG.

4.2.1.3 Lógica Reconfigurável (3,7% LUTs - 5.580 de 150.720)

A utilização de LUTs é notavelmente baixa (3,7%), evidenciando que o design concentra a computação em blocos dedicados (DSP48E1) ao invés de lógica genérica. Este padrão é desejável em projetos FPGA de alto desempenho, pois blocos especializados oferecem maior eficiência energética e throughput. A baixa utilização de LUTs (5.580 de 150.720) sugere margem para expansão de lógica de controle, máquinas de estado mais complexas ou hierarquias de ordenação adicionais sem comprometer recursos computacionais críticos.

4.2.1.4 Registradores (0,4% - 1.201 de 301.440)

O consumo mínimo de registradores (0,4%) indica pipeline enxuto e fluxo de dados direto entre módulos. Embora esta métrica demonstre eficiência, também sugere oportunidade para técnicas de pipeline mais profundo, que poderiam reduzir o caminho crítico através da inserção de estágios de registradores intermediários, aumentando a frequência máxima de operação para além dos 53,8 MHz atuais.

4.2.1.5 IOBs (87,2% - 523 de 600 pinos)

Os pinos de I/O representam o gargalo crítico do sistema, com 87,2% de utilização. Esta alta demanda decorre da arquitetura paralela de 16 elementos por ciclo, onde cada elemento de 16 bits requer múltiplos pinos para entrada/saída simultânea. A proximidade do limite (apenas 77 pinos remanescentes) inviabiliza replicação adicional do módulo Score sem redesenho fundamental da interface de comunicação.

4.2.1.6 Recursos de Clock (3,1% - 1 BUFG de 32)

O sistema utiliza apenas 1 buffer de clock global (BUFG), evidenciando design síncrono simples e robusto. A disponibilidade de 31 BUFGs adicionais permite implementação futura de múltiplos domínios de clock, útil para otimização de temporização em versões expandidas ou integração com interfaces de alta velocidade (DDR3, PCIe).

Conclusão: O design apresenta perfil assimétrico, com IOBs saturados (87,2%) enquanto recursos computacionais (DSP48E1: 16,9%) e de memória (BRAMs: 3,1%) permanecem subutilizados. Este padrão indica que o sistema está **limitado por largura**

de banda de I/O e não por capacidade computacional, direcionando otimizações futuras para técnicas de compressão de interface e multiplexação temporal.

4.3 Desempenho Computacional

4.3.1 Latência e Throughput

Com frequência de 50 MHz (período de 20 ns):

- **Latência por vetor (1024 dim):** 67 ciclos = $1,34 \mu\text{s}$
- **Throughput por módulo Score:** 1 score a cada $1,34 \mu\text{s}$
- **Taxa de operações:** 16 mult + 15 add = 31 ops/ciclo
- **GOPS (módulo base):** $31 \times 50 \text{ MHz} = 1,55 \text{ GOPS}$

Os 130 DSP48E1 utilizados indicam potencial para processamento paralelo além do módulo básico (que requer 32 DSPs), resultando em throughput agregado superior.

4.3.2 Comparação Hardware vs Software

Métrica	FPGA 50 MHz	CPU 2,5 GHz	Fator de Melhoria
Produto interno (1024 dim)	$1,2 - 1,5 \mu\text{s}$	$8,0 - 15,0 \mu\text{s}$	$5\times - 12\times$
Operações paralelas	20 – 50 ops/ciclo	2 – 8 ops/ciclo	$5\times - 25\times$
Jitter de Latência	$< 0,05 \mu\text{s}$	$5,0 - 20,0 \mu\text{s}$	$100\times - 400\times$

Tabela 7 – Comparação de desempenhos: FPGA vs CPU

O acelerador FPGA demonstra vantagens significativas em latência determinística e eficiência energética, mesmo operando a frequência $50\times$ inferior à CPU.

4.4 Sumário dos Resultados

O acelerador hardware k-NN implementado demonstra:

1. **Temporização robusta:** Margem de 7,0% a 50 MHz, sem erros em 1,8 bilhões de caminhos analisados
2. **Uso eficiente de recursos:** Concentração em DSP48E1 (16,9%) para computação intensiva, baixa utilização de lógica (3-5%)

3. **Alto desempenho:** Latência determinística de $1,34 \mu\text{s}$ por produto interno, speedup de $7,5\times$ sobre CPU
4. **Qualidade de design:** Síncrono puro, fanout médio 2,36, 100% roteado
5. **Escalabilidade limitada:** IOBs são gargalo (87,2%); com serialização de I/O, permitiria replicação $5,9\times$ limitada por DSP48E1

Os resultados validam a viabilidade e eficiência da arquitetura proposta para aceleração hardware de algoritmos k-NN em espaços de alta dimensionalidade.

4.5 Contribuições

Este trabalho apresentou o desenvolvimento e implementação de um acelerador hardware para o algoritmo k-NN em FPGA Xilinx Virtex-6 XC6VLX240T, demonstrando viabilidade técnica e vantagens significativas para aplicações em espaços de alta dimensionalidade (1024 dimensões).

A arquitetura modular proposta (Quantização, Referência, Score e Ordenação) alcançou resultados expressivos que validam a adequação de FPGAs para aceleração de algoritmos k-NN. Em termos de **determinismo e tempo real**, o sistema apresenta latência fixa de $1,34 \mu\text{s}$ por classificação (67 ciclos a 50 MHz), característica essencial para sistemas críticos onde variabilidade de tempo de resposta é inaceitável.

O **throughput competitivo** de 1,55 GOPS (31 operações por ciclo) demonstra speedup de $7,5\times$ sobre implementação sequencial em CPU, mantendo 100% de acurácia em relação à implementação de referência em ponto flutuante. A análise de utilização de recursos revela **margem significativa para escalabilidade**: blocos DSP48E1 consumindo apenas 16,9% indicam capacidade para $5,9\times$ replicação; BRAMs com 3,1% de ocupação suportam armazenamento de 2.000+ vetores de referência; LUTs utilizando 3,7% dos recursos disponíveis permitem expansão substancial da lógica de controle; e registradores com 0,4% de utilização viabilizam implementação de pipeline profundo para aumento de frequência de operação. A **qualidade de implementação** é evidenciada pelo slack de 1,402 ns (7% de margem de temporização), 100% de sinais roteados com sucesso e fanout médio de 2,36, indicando design robusto e escalável.

5 CONCLUSÃO

5.1 Limitação Identificada e Trabalhos Futuros

A análise de recursos identificou os **pinos de I/O como gargalo crítico** (87,2% de utilização), limitando expansão do sistema. Este gargalo decorre da arquitetura atual que transfere 16 elementos de 32 bits (float32) por ciclo, consumindo 523 dos 600 IOBs disponíveis.

Como propostas de trabalhos futuros, destacam-se:

- **Remoção do módulo Quantização:** Mudança da interface paralela de 16 float32 para 32 elementos em ponto fixo de 16 bits, simultaneamente aumentando throughput e reduzindo latência. Esta proposta representa o **maior potencial de melhoria identificado**;
- **Escalar para mais vetores de entrada:** Expandir a capacidade do sistema além dos 8 vetores atuais, explorando diferentes arquiteturas de memória e estratégias de pipeline para maximizar o número de vetores processados simultaneamente;
- **Desenvolver IP core reutilizável:** Criar um núcleo IP parametrizável que permita configuração flexível do número de vetores, dimensionalidade e formato de dados, facilitando integração em diferentes projetos e FPGAs;
- **Integração com sistemas RAG:** Implementar interfaces de comunicação (PCIe, AXI, Ethernet) para integração direta com frameworks de RAG existentes, permitindo uso do acelerador em aplicações reais de recuperação de contexto para Large Language Models.

5.2 Considerações Finais

Este trabalho demonstrou que FPGAs oferecem vantagens únicas para aceleração de k-NN: sistema de tempo real, eficiente em energia e escalável. O trabalho ainda tem espaço para grandes melhorias em latência e processamento com mais vetores. Os resultados obtidos contribuem para o estado da arte em aceleração hardware de algoritmos de aprendizado de máquina, demonstrando que designs especializados podem alcançar ordem de grandeza de melhoria mantendo acurácia de 100% e oferecendo garantias de tempo real.

Apêndices

APÊNDICE A – CONSUMO ENERGÉTICO DE LARGE LANGUAGE MODELS

Este apêndice apresenta dados detalhados sobre o consumo de eletricidade e emissões de CO_2 durante o treinamento de diversos Large Language Models (LLMs) desenvolvidos entre 2018 e 2024. Os dados foram compilados de Ji e Jiang (2026) e demonstram a evolução exponencial dos custos energéticos associados ao desenvolvimento de modelos cada vez maiores e mais sofisticados.

Desenvolvedor	Modelo	Ano Lanç.	Parâmetros (Bilhões)	Tipo de Acelerador	Consumo (MWh)	CO ₂ (Ton)
Google	BERT	2018	0.1	GPU V100	1.5	0.7
	GLaM	2021	1162	TPU v4	456	40
	PaLM	2022	540	TPU v4	3436	271.4
OpenAI	GPT-2	2019	1.5	TPU v3	1.7	0.7
	GPT-3	2020	175	GPU V100	1287	552
	GPT-4	2023	1800	GPU A100	51,772 to 62,318	12,456 to 14,994
Meta	OPT	2022	175	GPU A100	324	75
	LLaMA 2	2023	7	GPU A100	74	31.2
			13		147	62.4
			70		688	291.4
			405		/	8930
	LLaMA 3	2024	1	GPU H100	/	107
			3		/	133
			8		/	390 to 420
			70		/	1900 to 2040
			405		/	8930
BigScience	BLOOM	2022	1	GPU A100	159	9.1
			104		267	15.2
			176		433	24.7
DeepMind	Gopher	2021	280	TPU v3	1151	380
			/		1066	352
THI	Falcon	2023	40	GPU A100	163	8 to 90
			180		2,940	147 to 1,617
Mistral AI	Mixtral 8 × 7B	2024	47	GPU A100	232	6.3
DeepSeek	DeepSeek-V3	2024	671	GPU H800	1,025	550

Tabela 8 – Parâmetros e avaliações de consumo de eletricidade para vários LLMs líderes

Fonte: Adaptado de (JI; JIANG, 2026), Tabela 3

Observações:

- Os valores de consumo energético referem-se ao treinamento completo dos modelos;
- Emissões de CO_2 variam de acordo com a matriz energética da região onde o treinamento foi realizado;

- Modelos mais recentes (2023-2024) utilizam aceleradores de última geração (GPU H100, H800) que são mais eficientes, mas o número massivo de parâmetros resulta em consumo total ainda maior;
- O símbolo “/” indica dados não disponíveis na fonte original.

APÊNDICE B – ANÁLISE ESTATÍSTICA DOS DATASETS DA COHERE

Este apêndice apresenta a análise estatística dos datasets de embeddings da Cohere utilizados como dados de entrada no sistema proposto ((conforme citado na subseção 3.3.1), incluindo o dataset wikipedia-22-12-en-embeddings citado 1.

```

1  #!/usr/bin/env python3
2  """
3  Stats for Vectorial Databases in Apache Parquet
4
5  Copyright (c) 2025, Augusto Damasceno.
6  All rights reserved.
7
8  SPDX-License-Identifier: BSD-2-Clause
9  """
10
11  __author__ = "Augusto Damasceno"
12  __version__ = "2.0"
13  __copyright__ = "Copyright (c) 2025, Augusto Damasceno."
14  __license__ = "BSD-2-Clause"
15
16  from pathlib import Path
17  from typing import Optional, List
18  import sys
19
20  import numpy as np
21  from pyarrow import Table
22  from pyarrow.parquet import ParquetFile
23  from pyarrow import concat_tables
24
25
26  MAX_BYTES = 1024 ** 3
27  DATASET_DIR = Path.home() / "datasets" / "wikipedia-22-12-en-
    embeddings"
28
29
30  def get_dataset_paths(base_dir: Optional[Path] = None) -> List[
    Path]:

```

```
31     """
32     Returns list of all parquet files in the dataset directory.
33
34     Args:
35         base_dir: Base directory containing parquet files.
36                   Defaults to ~/datasets/wikipedia-22-12-en-
37                   embeddings
38
39     Returns:
40         Sorted list of Path objects for all .parquet files
41     """
42     if base_dir is None:
43         base_dir = DATASET_DIR
44
45     if not base_dir.exists():
46         print(f"Error: Dataset directory not found: {base_dir}",
47               file=sys.stderr)
48         return []
49
50     parquet_files = sorted(base_dir.glob("*.parquet"))
51
52     if not parquet_files:
53         print(f"Warning: No parquet files found in {base_dir}",
54               file=sys.stderr)
55
56     return parquet_files
57
58 def read_parquet_head(file_path: Path, max_bytes: int = MAX_BYTES
59                       ) -> Optional[Table]:
60     """
61     Reads Parquet file up to byte limit.
62
63     Args:
64         file_path: Path to Parquet file
65         max_bytes: Maximum bytes to read
66
67     Returns:
68         Combined PyArrow table or None if error
69     """
70     if not file_path.exists():
```

```
68         print(f"Error: File not found: {file_path}", file=sys.
69               stderr)
70         return None
71
72     try:
73         parquet_file = ParquetFile(file_path)
74         tables = []
75         bytes_read = 0
76
77         for i in range(parquet_file.num_row_groups):
78             table = parquet_file.read_row_group(i)
79             if bytes_read + table.nbytes > max_bytes:
80                 break
81             tables.append(table)
82             bytes_read += table.nbytes
83
84         return concat_tables(tables) if tables else None
85
86     except Exception as e:
87         print(f"Error reading {file_path}: {e}", file=sys.stderr)
88         return None
89
90 def compute_embedding_stats(embeddings: np.ndarray) -> dict:
91     """
92     Computes embedding statistics.
93
94     Args:
95         embeddings: Numpy array with embeddings (NxD)
96
97     Returns:
98         Dictionary with computed statistics
99     """
100     flat_data = embeddings.ravel()
101     diff = np.diff(flat_data)
102
103     return {
104         'min': embeddings.min(),
105         'max': embeddings.max(),
106         'std_diff': np.std(np.abs(diff)),
107         'shape': embeddings.shape,
```

```

108         'total_elements': embeddings.size
109     }
110
111
112 def analyze_range_distribution(embeddings: np.ndarray, total: int
113     ) -> None:
114     """
115     Analyzes distribution of values outside specific ranges.
116
117     Args:
118         embeddings: Numpy array with embeddings
119         total: Total number of elements
120     """
121     print("\n=== Values outside range ===")
122
123     # Fractional intervals [0.1, 1.0]
124     for i in range(1, 11):
125         value = i / 10
126         outside = np.sum((embeddings > value) | (embeddings < -
127             value))
128         pct = outside / total
129         print(f"Outside [{-value:.1f}, {value:.1f}]: {pct:.4%} ({
130             outside:,,} of {total:,,})")
131
132     # Integer intervals [1, 9]
133     for value in range(1, 10):
134         outside = np.sum((embeddings > value) | (embeddings < -
135             value))
136         pct = outside / total
137         print(f"Outside [{-value}, {value}]: {pct:.4%} ({outside
138             :,,} of {total:,,})")
139
140
141 def analyze_inclusion_distribution(embeddings: np.ndarray, total:
142     int) -> None:
143     """
144     Analyzes distribution of values within specific ranges.
145
146     Args:
147         embeddings: Numpy array with embeddings
148         total: Total number of elements

```

```

143     """
144     print("\n=== Values within range ===")
145
146     for i in range(20):
147         limit = i / 10
148         inside = np.sum((embeddings >= -limit) & (embeddings <=
149             limit))
150         pct = inside / total
151         print(f"Between [{-limit:.1f}, {limit:.1f}]: {pct:.4%} ({
152             inside:.,} of {total:.,})")
153
154 def process_dataset(dataset_path: Path) -> None:
155     """
156     Processes a dataset and displays its statistics.
157
158     Args:
159         dataset_path: Path to dataset file
160     """
161     print(f"\n{'=' * 70}")
162     print(f"Processing: {dataset_path.name}")
163     print(f"{'=' * 70}")
164
165     data = read_parquet_head(dataset_path)
166     if data is None:
167         return
168
169     # Convert to numpy array
170     embeddings = np.stack(data['emb'].to_numpy())
171
172     # Basic statistics
173     stats = compute_embedding_stats(embeddings)
174     print(f"\nBasic statistics:")
175     print(f"    Shape: {stats['shape']}")
176     print(f"    Total elements: {stats['total_elements']:.,}")
177     print(f"    Min: {stats['min']:.6f}")
178     print(f"    Max: {stats['max']:.6f}")
179     print(f"    Std(|diff|): {stats['std_diff']:.6f}")
180
181     # Distribution analyses
182     analyze_range_distribution(embeddings, stats['total_elements']

```

```
    ])  
182     analyze_inclusion_distribution(embeddings, stats['  
        total_elements'])  
183  
184  
185 def main() -> int:  
186     """Main function."""  
187     try:  
188         dataset_paths = get_dataset_paths()  
189  
190         if not dataset_paths:  
191             print("No datasets to process.", file=sys.stderr)  
192             return 1  
193  
194         print(f"Found {len(dataset_paths)} parquet file(s) to  
            process")  
195  
196         for path in dataset_paths:  
197             process_dataset(path)  
198  
199         return 0  
200  
201     except KeyboardInterrupt:  
202         print("\n\nInterrupted by user.", file=sys.stderr)  
203         return 130  
204     except Exception as e:  
205         print(f"\nFatal error: {e}", file=sys.stderr)  
206         return 1  
207  
208  
209 if __name__ == "__main__":  
210     sys.exit(main())
```

Listing B.1 – Análise Estatística dos Datasets

REFERÊNCIAS

ABDELRASOUL, M.; SHABAN, A. S.; ABDEL-KADER, H. FPGA Based Hardware Accelerator for Sorting Data. In: *International Japan-Africa Conference on Electronics, Communications, and Computations (JAC-ECC)*. [S.l.]: IEEE, 2021. p. 57–60.

Amazon Web Services. *Amazon Kendra: Intelligent Search Service*. 2023. <https://aws.amazon.com/kendra>. Accessed: 2025-12-05.

Amazon Web Services. *What is RAG (Retrieval-Augmented Generation)?* 2024. <https://aws.amazon.com/what-is/retrieval-augmented-generation/>. Accessed: 2025-12-07.

Amazon Web Services. *Amazon EC2 F1 Instances: FPGA-Based Compute Instances for Accelerated Workloads*. 2025. <https://aws.amazon.com/ec2/instance-types/f1/>. Accessed: 2025-12-07.

Apache Software Foundation. *PyArrow: Python library for Apache Arrow*. 2023. <https://arrow.apache.org/docs/python/>. Python bindings for Apache Arrow. Accessed: 2025-12-08.

Chroma. *Chroma: The AI-native open-source embedding database*. 2023. <https://www.trychroma.com>. Accessed: 2025-12-05.

Cohere AI. *Introducing Embed v3: The First Embedding Model Built for Retrieval-Augmented Generation*. 2023. <https://cohere.com/blog/introducing-embed-v3>. Accessed: 2025-11-22.

Cohere AI. *wikipedia-22-12-en-embeddings: Wikipedia Embeddings for Cohere Embed v3*. [S.l.]: Hugging Face, 2023. <https://huggingface.co/datasets/Cohere/wikipedia-22-12-en-embeddings>. Accessed: 2025-11-22.

Deepset. *Haystack: Open-source NLP Framework*. 2023. <https://haystack.deepset.ai>. Accessed: 2025-12-05.

DIAS, L. A. et al. A full-parallel implementation of self-organizing maps on hardware. *Neural Networks*, Elsevier Ltd., 2021.

Elastic. *Elasticsearch: Search and Analyze Data in Real Time*. 2023. <https://www.elastic.co/elasticsearch>. Accessed: 2025-12-05.

Facebook AI. *FAISS: Efficient Similarity Search and Clustering of Dense Vectors*. 2023. <https://github.com/facebookresearch/faiss>. Accessed: 2025-12-05.

Google Cloud. *Google Vertex AI Search and Conversational AI*. 2023. <https://cloud.google.com/generative-ai-studio>. Accessed: 2025-12-05.

IBM. *O que são alucinações de IA?* 2024. <https://www.ibm.com/br-pt/think/topics/ai-hallucinations>. Accessed: 2025-12-07.

IEEE. *IEEE Standard for Floating-Point Arithmetic*. New York, NY, USA, 2019. 1–84 p.

- Intel Corporation. *BFLOAT16 Hardware Numerics Definition*. [S.l.], 2018. White Paper. Document Number: 338302-001US.
- JEGHAM, N. et al. *How Hungry is AI? Benchmarking Energy, Water, and Carbon Footprint of LLM Inference*. 2025. <https://arxiv.org/abs/2505.09598>.
- JI, Z.; JIANG, M. A systematic review of electricity demand for large language models: evaluations, challenges, and solutions. *Renewable and Sustainable Energy Reviews*, Elsevier, v. 225, p. 116159, 2026.
- JUNG, M. et al. Vector similarity search acceleration using dram-based processing-in-memory (pim). In: *2025 International Conference on Electronics, Information, and Communication (ICEIC)*. [S.l.]: IEEE, 2025. ISBN 979-8-3315-1075-6.
- LangChain. *LangChain: Building Applications with LLMs through Composability*. 2023. <https://github.com/langchain-ai/langchain>. Accessed: 2025-12-05.
- LEWIS, P. et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. In: *Advances in Neural Information Processing Systems*. [S.l.: s.n.], 2020. v. 33, p. 9459–9474.
- LlamaIndex (formerly GPT Index). *LlamaIndex: A Data Framework for LLM Applications*. 2023. <https://www.llamaindex.ai>. Accessed: 2025-12-05.
- MEYER-BAESE, U. *Digital Signal Processing with Field Programmable Gate Arrays*. 4th. ed. Berlin, Heidelberg: Springer, 2014. ISBN 978-3-642-45308-3.
- Microsoft. *Azure Cognitive Search: Cloud Search Service*. 2023. <https://learn.microsoft.com/en-us/azure/search/>. Accessed: 2025-12-05.
- MongoDB Inc. *MongoDB Atlas Vector Search*. 2023. <https://www.mongodb.com/products/platform/atlas-vector-search>. Accessed: 2025-12-05.
- NumPy Developers. *NumPy Documentation*. 2024. <https://numpy.org/doc/stable/>. Accessed: 2025-12-08.
- OpenAI. *OpenAI API: GPT-4 and Embeddings*. 2023. <https://platform.openai.com>. Accessed: 2025-12-05.
- Pinecone Systems Inc. *Pinecone: The Vector Database for AI*. 2023. <https://www.pinecone.io>. Accessed: 2025-12-05.
- Python Software Foundation. *Python Programming Language*. 2024. <https://www.python.org/>. Accessed: 2025-12-08.
- Qdrant. *Qdrant: Vector Database for High-Dimensional Data*. 2023. <https://qdrant.tech>. Implemented in Rust with HNSW and DiskANN algorithms. Accessed: 2025-12-05.
- Redis Inc. *Redis Stack: Vector Database and Search Capabilities*. 2023. <https://redis.io/docs/stack>. Redis with vector search module using HNSW for high-performance similarity search. Accessed: 2025-12-05.
- SeMI Technologies. *Weaviate: Vector Database*. 2023. <https://weaviate.io>. Accessed: 2025-12-05.

Stanford University. *DSPy: Declarative Self-Improving Language Programs*. 2023. <https://github.com/stanfordnlp/dspy>. Accessed: 2025-12-05.

SurrealDB. *SurrealDB: Multi-Model Database with Vector Support*. 2023. <https://surrealdb.com>. Open-source Rust-based database with vector and document capabilities. Accessed: 2025-12-05.

TEAM, T. pandas development. *pandas-dev/pandas: Pandas*. [S.l.]: Zenodo, 2020. <https://pandas.pydata.org/>. Accessed: 2025-12-08.

The MathWorks Inc. *MATLAB: The Language of Technical Computing*. Natick, Massachusetts, 2024. <https://www.mathworks.com/products/matlab.html>. Accessed: 2025-12-07.

The MathWorks Inc. *Simulink: Simulation and Model-Based Design*. Natick, Massachusetts, 2024. <https://www.mathworks.com/products/simulink.html>. Accessed: 2025-12-07.

Vald Community. *Vald: Distributed Vector Database*. 2023. <https://vald.vdaas.org>. Open-source distributed vector database with NGT and HNSW algorithms. Accessed: 2025-12-05.

Xilinx Inc. *ISE Design Suite 14.3: PlanAhead Software*. [S.l.], 2012. Accessed: 2025-12-07.

Xilinx Inc. *ML605 Hardware User Guide*. [S.l.], 2012. UG534 (v1.9).

Xilinx Inc. *Xilinx System Generator for DSP User Guide*. 2012. https://www.xilinx.com/support/documentation/sw_manuals/xilinx14_3/sysgen_user.pdf. Accessed: 2025-12-07.

Xilinx Inc. *Bitonic Sort*. [S.l.], 2019. Documentação da biblioteca Vitis Database - Algoritmo Bitonic Sort para FPGAs.

Yahoo Inc. *Vespa: Open-Source Data and ML Serving Platform*. 2023. <https://vespa.ai>. Supports ANN with HNSW and LSH. Accessed: 2025-12-05.

Zilliz. *Milvus: Open-source Vector Database*. 2023. <https://milvus.io>. Accessed: 2025-12-05.